



UNIVERSIDAD DEL BÍO-BÍO, CHILE

FACULTAD DE CIENCIAS EMPRESARIALES

Departamento de Sistemas de Información

"PLATAFORMA WEB DE ADMINISTRACIÓN VISUAL DE INFRAESTRUCTURA TI"

PROYECTO DE TÍTULO PARA OPTAR AL TÍTULO DE
INGENIERO DE EJECUCIÓN EN COMPUTACIÓN E INFORMÁTICA

BENJAMÍN ALEJANDRO GUAJARDO HERRERA

DIRIGIDA POR

PROFESORA MÓNICA ALEJANDRA CANIUPÁN MARILEO

2025

Resumen

La creciente complejidad de la infraestructura tecnológica en organizaciones públicas y privadas ha incrementado la necesidad de herramientas que permitan supervisar, administrar y automatizar procesos operativos de forma eficiente. En la actualidad, gran parte de los dispositivos y servicios críticos dependen de monitoreo continuo para asegurar la disponibilidad, la seguridad y la estabilidad de las operaciones. Sin embargo, las soluciones tradicionales presentan limitaciones en cuanto a usabilidad, fragmentación de funcionalidades, sobrecarga de alertas y dependencia de configuraciones manuales, lo que dificulta la toma de decisiones oportuna por parte de los equipos técnicos.

Este proyecto propone el desarrollo de Nocturne Scope, una plataforma web diseñada para centralizar el monitoreo de infraestructura TI mediante una visualización gráfica basada en nodos, la recolección de métricas en tiempo real y un sistema modular de automatización operativa. La propuesta integra en un único entorno funcionalidades que habitualmente se encuentran dispersas en múltiples herramientas, tales como visualización de topologías, generación de reportes, registro de eventos y ejecución de tareas automáticas.

La solución está orientada a profesionales del área de infraestructura TI, especialmente a equipos con niveles intermedios de experiencia, que requieren herramientas intuitivas, eficientes y con baja curva de aprendizaje. Su enfoque incremental permite validar cada módulo de forma progresiva, asegurando estabilidad y compatibilidad en la integración de tecnologías como Go, Next.js, PostgreSQL e InfluxDB.

En este contexto, el presente informe documenta el análisis, diseño, implementación y validación de la plataforma, abarcando desde la identificación de la problemática hasta la demostración de viabilidad técnica, operativa y económica. Con ello, se busca aportar una alternativa moderna y accesible para mejorar la continuidad operativa, reducir los tiempos de respuesta y fortalecer la resiliencia tecnológica de las organizaciones.

Palabras Clave — Monitoreo de infraestructura TI, Topología, Automatización de Tareas, Administración de Dispositivos, Redes Computacionales.

Agradecimientos

Hay personas que se vuelven o siempre han sido el eje central de tu vida, pilares fundamentales que permiten que las caídas que puedas tener no sean tan dolorosas, te ayudan a levantarte paso a paso, a tu ritmo, respirar, alcanzar una nueva vida; Nueva vida, eso es lo que mi madre Marcela Herrera y mi padre Juan Guajardo me dieron, porque a pesar que me vieron en el pozo, ahogandome sin remedio, hicieron lo imposible para que pudiera seguir, los amaré por siempre y mi corazón les pertenecerá plenamente a ellos.

Quiero agradecer también a mi hermano, mi amigo de la vida, esa persona con la que puedo hablar de mis hiperfijaciones y me es recíproco. Él a alimentado mi hambre de conocimiento, me ha guiado para poder darle sentido a todo lo que quiero, a mis metas. Espero y haré el eterno esfuerzo de que nunca perdamos nuestra cercanía, porque una vida si ti carecería de todo el sentido.

También quiero destacar la labor de los Académicos de la Universidad del Bío-bío, en especial al profesor Luis Cabrera y a la profesora Mónica Caniupán; Les agradezco por enseñarme el ARTE que es la informática, como a través de algo tan abstracto podemos crear realidades.

Por último pero no menos importante quiero agradecer al profesor Álvaro Alarcón, por ser mi mentor este último año y abrirme los ojos a un mundo lleno de posibilidades que espero seguir explorando mi vida entera.

De verdad gracias a todos, y como diría mi musical favorito que me ha inspirado todo este mes **I am not throwing away my shot**, esto solo es el inicio.

Acrónimos

CPU: Central Processing Unit (Unidad Central de Procesamiento).

GPU: Graphics Processing Unit (Unidad de Procesamiento Gráfico).

HDD: Hard Disk Drive (Unidad de Disco Duro).

IP: Internet Protocol (Protocolo de Internet).

RAM: Random Access Memory (Memoria de Acceso Aleatorio).

SSD: Solid State Drive (Unidad de Estado Sólido).

TCP/IP: Transmission Control Protocol/Internet Protocol (Protocolo de Control de Transmisión/Protocolo de Internet).

TI: Tecnologías de la Información.

VPS: Virtual Private Server (Servidor Privado Virtual).

Índice General

Acrónimos	IV
1. Introducción	1
1.1. Presentación del área y su problemática	1
1.1.1. Presentación de la Empresa	1
1.1.2. Presentación de los procesos del ámbito del proyecto en la Empresa (proceso de negocio)	2
Presentación de los procesos del ámbito del proyecto	2
1.1.3. Descripción del Problema	3
1.2. Propuesta de Solución	4
1.3. Análisis de los Principales Trabajos Realizados o soluciones disponible en el área o tema de la propuesta	6
1.4. Justificación de la Propuesta	8
1.5. Objetivos del proyecto	9
1.5.1. Objetivo General	9
1.5.2. Objetivos Específicos	9
1.6. Composición del Informe	10
2. Proyecto	11
2.1. Objetivos del Software	11
2.1.1. Objetivo General	11
2.1.2. Objetivos Específicos	11
2.2. Metodología de Desarrollo	12
2.3. Tecnologías Utilizadas (Backend, Frontend, Base de Datos)	13
2.4. Estándares de Documentación	14
2.5. Técnicas y notaciones	14
2.6. Factibilidad del Proyecto	15
2.6.1. Factibilidad Técnica	15
2.6.2. Factibilidad Operativa	16
2.6.3. Factibilidad Económica	17
Flujo de caja	18
Cálculo del V.A.N	19

2.6.4. Conclusión de Factibilidad	19
3. Especificación de Requerimientos - Producto de Software	20
3.1. Límites	20
3.2. Rectricciones Técnicas	21
3.3. Requerimientos Funcionales	22
3.4. Requerimientos No Funcionales	24
3.5. Interfaces externas de Entrada	26
3.6. Interfaces externas de Salida	26
4. Análisis - Casos de Uso	27
4.1. Historias de usuario	28
4.2. Actores de casos de uso	29
4.3. Diagramas y Especificación de casos de uso	30
4.3.1. Casos de Uso: Dispositivo	30
4.3.2. Casos de Uso: Reporte	33
4.3.3. Casos de Uso: Topología	35
5. Diseño	40
5.1. Descripción de los servicios web - necesarios	40
5.2. Modelo de datos	41
5.2.1. Modelo Entidad-Relación	41
5.2.2. Modelo Relacional	42
5.3. Diseño de interfaz y navegación (Mockups)	43
5.3.1. Guías de estilos	43
5.3.2. Logotipo	43
5.3.3. Guía de colores	44
5.3.4. Tipografía	45
5.3.5. Composición de las interfaces	46
6. Desarrollo del Trabajo	49
6.1. Diseño de arquitectura	49
6.2. Estructura del código	50
Clean Architecture	50
6.2.1. Backend	53
Funcionalidad-Endpoints a utilizar:	54
6.2.2. Frontend	57
7. Implantación y Puesta en Marcha	58
7.1. Plan de Capacitación/Entrenamiento	58
7.2. Estrategia de Implantación	60

8. Conclusiones	63
8.1. Trabajo Futuro	64
Referencias	65
A. Definiciones y abreviaciones del Negocio	67
B. Recopilación de Información	69
C. Diccionario de datos	72
D. Aspectos de gestión de proyectos	76
D.1. Carta Gantt con línea base y desviaciones	76
D.2. Riesgos de Alto nivel (Amenazas), Impacto, estrategia	78
D.3. Estimación CU	79
D.4. Resumen Esfuerzo	82
D.5. Retrospectiva Proyecto	82
D.6. Iteraciones en el desarrollo	84

Índice de Figuras

1.1. BPMN Situación Actual.	3
1.2. Representación de Dispositivos en Red en forma de Nodos.	4
1.3. Representación de Automatización de Notificaciones en un dispositivo Linux.	5
2.1. Diagrama Metodología Evolutiva	12
4.1. Diagrama de Casos de Uso Dispositivo	30
4.2. Diagrama de Casos de Uso Reporte	33
4.3. Diagrama de Casos de Uso Reporte	35
5.1. Diagrama Entidad-Relación	41
5.2. Diagrama Entidad-Relación	42
5.3. Logo Nocturne Security	43
5.4. Logo Nocturne Scope	43
5.5. Paleta Rosé Pine	44
5.6. Tipografía	45
5.7. Landing Page	46
5.8. Dashboard	47
5.9. Topología	48
6.1. Arquitectura en Red del Proyecto	50
6.2. Diagrama Clean Architecture	51
7.1. Cronograma referencial de implementación del sistema durante dos meses (enero y febrero del próximo año). Las columnas 1 a 8 representan semanas consecutivas.	62
D.1. Carta Gantt con Desviaciones (Ejecución Real)	77

Índice de Tablas

2.1. Especificación de Hardware requerido en desarrollo del proyecto	15
2.2. Especificación de Hardware Servidor requerido en desarrollo del proyecto	16
2.3. Licencias del software utilizado en el desarrollo del proyecto	17
2.4. Costo hosting y dominio	17
2.5. Cálculo de costo de desarrollo y soporte basado en sueldo Estudiante y Desarrollador Junior	17
2.6. Calculo costo de desarrollo y soporte.	18
2.7. Cálculo del VAN con tasa del 10 %	19
3.2. Requerimientos Funcionales	24
3.4. Requerimientos No Funcionales	25
3.5. Interfaces de Entrada	26
3.6. Interfaces de Salida	26
4.1. Historias de Usuario.	28
4.2. Actores de casos de usos.	29
6.1. Estructura del Backend	53
6.2. Estructura del Frontend	57
B.1. Resumen de fuentes documentales revisadas.	69
B.2. Ficha de hallazgos mediante observación directa.	71
C.1. Entidad: Usuarios (users)	72
C.2. Entidad: Dispositivos (devices)	73
C.3. Entidad: Topologías (topologies)	73
C.4. Entidad: Tokens de Acceso (api_tokens)	74
C.5. Medición: Telemetría del Sistema (system_metrics)	74
C.6. Medición: Tráfico de Red (network_traffic)	75
D.1. Matriz de Riesgos, Estrategias y Estado Final	78
D.2. Cálculo estimación tipo de caso de uso.	79
D.3. Cálculo estimación tipo de actor.	79

D.4. Evaluación de relevancia de factores técnicos.	80
D.5. Factores de complejidad técnica.	80
D.6. Factores ambientales.	81
D.7. Anexo resumen esfuerzo.	82
D.8. Porcentaje de cumplimiento de requerimientos por módulo.	83
D.9. Resumen de iteraciones, funcionalidades y validaciones del proyecto.	85

Capítulo 1

Introducción

El crecimiento acelerado de la digitalización en empresas y organismos públicos ha incrementado de forma significativa la dependencia sobre la infraestructura tecnológica. En este escenario, los profesionales encargados de administrar redes, servidores y servicios críticos asumen un rol fundamental para garantizar la continuidad operativa y la seguridad de los sistemas. Sin embargo, la complejidad creciente de las redes modernas, junto con la persistencia de procesos manuales y la limitada capacidad de las herramientas tradicionales de monitoreo, ha generado nuevos desafíos que afectan tanto la eficiencia operativa como la capacidad de respuesta ante incidentes. Este capítulo presenta el contexto general de la infraestructura TI, describe los procesos involucrados y expone las principales problemáticas que motivan el desarrollo de una solución que permita mejorar la supervisión, visibilidad y automatización dentro de entornos tecnológicos actuales.

1.1. Presentación del área y su problemática

1.1.1. Presentación de la Empresa

La siguiente propuesta de solución está destinada a los profesionales del área de Infraestructura de las Tecnologías de la Información (TI), de cualquier empresa, quienes son los responsables de las operaciones diarias de los dispositivos y servicios tecnológicos de una empresa. Entre las principales labores desempeñadas por el personal encargado, se encuentra la administración de la red, servidores y servicios en la nube, garantizando la conectividad entre redes y el acceso seguro de usuarios y sistemas.

La importancia central de la labor desempeñada por estos profesionales radica en garantizar la continuidad y disponibilidad de la infraestructura tecnológica, especialmente en el contexto actual, donde una proporción significativa de los procesos se han digitalizado o se encuentran en vías de hacerlo. Como se desprende del informe de las Naciones Unidas (ONU) [1] sobre el desarrollo de los gobiernos digitales, casi un 72 % de los países evaluados poseen niveles de digitalización «muy alto» y «alto». Este dato resulta de particular interés, ya que arroja indicios sobre la situación de la digitalización en el sector público. En contraste con lo anterior, el sector

privado, de acuerdo con el Foro Económico Mundial en su informe "Jobs Report-[2] , indica que un 85 % de las empresas encuestadas se encuentran en un proceso de implementación de nuevas tecnologías, replicable para el resto de las empresas a nivel global.

En consecuencia, se puede afirmar que, durante el proceso de migración de los procedimientos, el refuerzo de la infraestructura que los respalda adquiere una relevancia fundamental. En este sentido, se torna imperativo identificar soluciones que respalden la labor de los profesionales del área.

1.1.2. Presentación de los procesos del ámbito del proyecto en la Empresa (proceso de negocio)

Presentación de los procesos del ámbito del proyecto

Los procesos del área de Infraestructura TI relevantes para este proyecto corresponden al monitoreo y la administración de redes. Estas actividades son esenciales para asegurar la continuidad operativa de la organización, ya que permiten mantener la conectividad, garantizar el rendimiento de los sistemas y proteger la seguridad de la información que transita por la infraestructura tecnológica.

Dentro las labores más importantes que desempeñan los profesionales del área podemos destacar:

- **Supervisión permanente de dispositivos de red**, como lo son servidores físicos y virtuales. Este proceso permite anticipar o detectar caídas, disminución de rendimiento y fallas críticas que, de no ser atendidas a tiempo, pueden impactar directamente en la disponibilidad de los servicios.
- **Gestión de incidentes y alertas**, que contempla la identificación, clasificación y resolución de problemas en la infraestructura, es un aspecto de suma importancia para garantizar la continuidad del negocio.
- **Administración y cambios de configuraciones de los dispositivos en la red**, tales como servidores y ordenadores portátiles, entre otros. Esta actividad, de naturaleza continua, conlleva la gestión de parámetros de red, la implementación de políticas de seguridad y la coordinación de modificaciones en diversos dispositivos o entornos.

Estas funciones forman parte del alcance del proyecto y servirán de base para la elaboración de una propuesta de solución orientada a enfrentar las principales problemáticas que surgen en el área, las cuales se presentarán en la siguiente sección **1.1.3. Descripción del Problema**.

1.1.3. Descripción del Problema

La digitalización de procesos en la industria a aumentado significativamente la dependencia de la infraestructura tecnológica. Sin embargo, este desarrollo ha traído consigo una mayor complejidad en la gestión de redes y un aumento en la exposición a fallas en los servicios virtuales de las empresas, errores de configuración en dispositivos conectados a la red y ciberataques. El informe "The Many Costs of Downtime"[3] de la empresa global de tecnología de redes informática, Opendgear, muestra que más del 60 % de las interrupciones de red generan pérdidas superiores a los 100.000 dólares, mientras que un 15 % supera el millón. Además, las organizaciones tardan en promedio 11 horas en identificar y resolver un incidente, prolongando la indisponibilidad y sus efectos financieros y reputacionales.

A esta problemática se suma la persistencia de procesos manuales en la administración de redes. Según el informe "Network Automation Trends"[4] de Cisco, hasta un 95 % de los cambios de configuración se siguen realizando de forma manual, lo que incrementa la probabilidad de errores humanos, genera inconsistencias operativas y limita la capacidad de respuesta frente a incidentes. La falta de automatización no solo reduce la eficiencia de los equipos técnicos, sino que también encarece las operaciones y retrasa la implementación de medidas preventivas.

Otro punto relevante a tomar en cuenta es que el diseño de sistemas de monitoreo puede convertirse en un obstáculo para la toma de decisiones si no considera adecuadamente los requerimientos de los profesionales en redes. El estudio "Nationwide Critical Infrastructure Monitoring"[5] de S. Puusk, evidencia que un diseño deficiente provoca sobrecarga de información y limita la capacidad de los operadores para mantener una visión común del estado de la red, lo que reduce la efectividad de la respuesta frente a incidentes.

En síntesis, el problema radica en los altos costos asociados a interrupciones, la dependencia de procesos manuales que aumentan los riesgos, la insuficiencia de las herramientas de monitoreo tradicionales, las limitaciones de las soluciones actualmente implementadas y los problemas de diseño que afectan la eficiencia de los sistemas. Estas relaciones y flujos se observan en el diagrama presentado en la Figura 1.1, donde se detallan los puntos críticos del proceso.

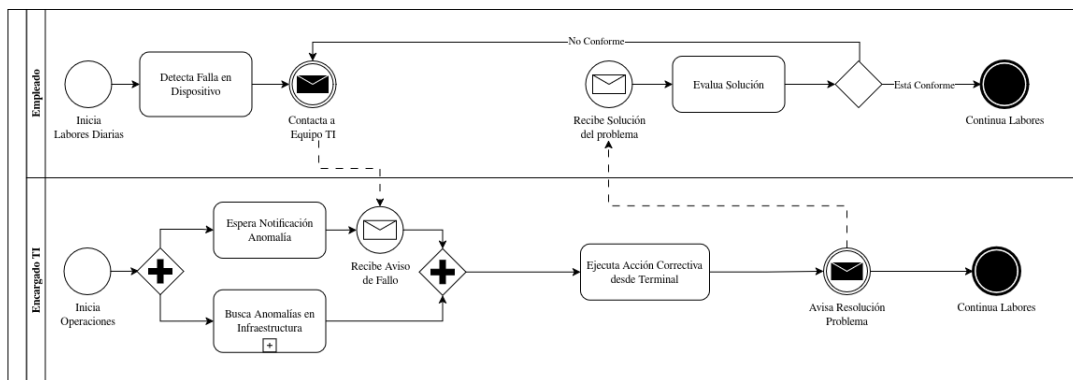


Figura 1.1: BPMN Situación Actual.

1.2. Propuesta de Solución

La solución propuesta consiste en el desarrollo de una aplicación de administración de infraestructura TI con un enfoque centrado en la visualización y representación gráfica de los dispositivos conectados a la red. A diferencia de herramientas tradicionales, este sistema se caracterizará por mostrar los elementos de red a través de un esquema gráfico de nodos como se aprecia en la Figura 1.2, estos representarán mediante flechas y conexiones la relación entre dispositivos. Cada nodo permite observar información general del equipo, como el nivel de uso de procesador, capacidad gráfica o temperatura, empleando indicadores visuales fáciles de interpretar que faciliten una rápida comprensión del estado de la red.

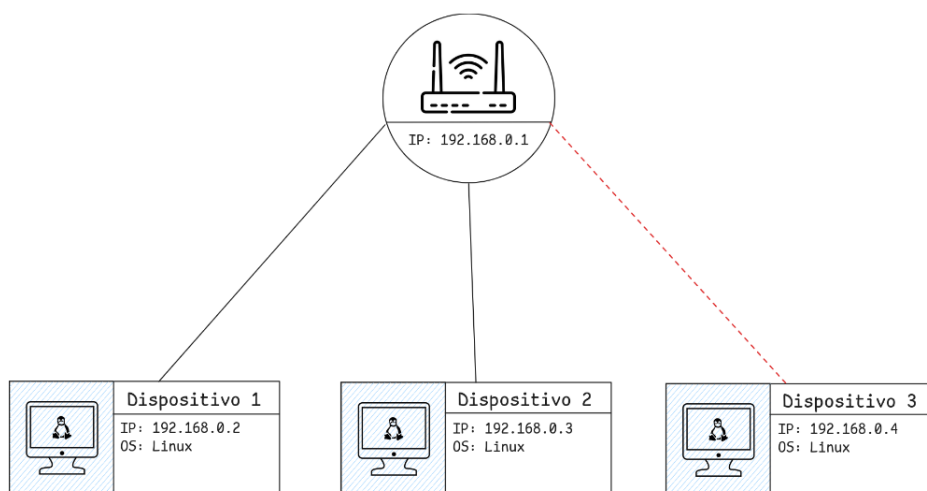


Figura 1.2: Representación de Dispositivos en Red en forma de Nodos.

La plataforma funciona como aplicación web, lo que posibilita su despliegue tanto en entornos locales como en la nube. En una primera etapa se prioriza la integración de servidores y dispositivos Linux, físicos y virtuales, al ser componentes esenciales en la creación de redes empresariales.

Uno de los principales elementos diferenciador de este software será la incorporación de mecanismos de automatización de tareas de forma gráfica y accesible. Esto significa que el sistema no solo permite detectar eventos, como un uso excesivo del procesador o la caída de un enlace, sino también asociarles respuestas automáticas previamente definidas. Entre ellas, se contempla el envío de notificaciones por aplicaciones de mensajería y correo electrónico, garantizando que los equipos responsables reciban alertas oportunas y puedan actuar de inmediato, tal y como se puede apreciar en la Figura 1.3.

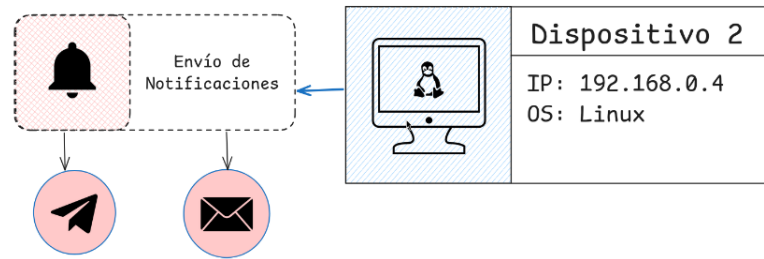


Figura 1.3: Representación de Automatización de Notificaciones en un dispositivo Linux.

Con este diseño, la aplicación busca resolver problemas comunes en el ámbito de la administración TI, como la falta de visibilidad integral, la sobrecarga de alertas poco relevantes y la dependencia de procedimientos manuales. Al reunir en un mismo entorno la supervisión visual y la automatización de acciones, se espera reducir los tiempos de detección y recuperación de fallas, mejorar la eficiencia de los equipos de soporte y, en última instancia, fortalecer la continuidad operativa de las organizaciones.

1.3. Análisis de los Principales Trabajos Realizados o soluciones disponible en el área o tema de la propuesta

Con el objetivo de identificar herramientas ya existentes para el monitoreo y la gestión de infraestructura tecnológica, se realizó una búsqueda bibliográfica y técnica durante el periodo comprendido entre el 01 al 15 de Septiembre del 2025. A partir de esta revisión, se seleccionaron las siguientes aplicaciones, las cuales destacan por su impacto y uso extendido en entornos empresariales.

- **Nagios [6]** : Siendo una de las herramientas de monitoreo más veteranas, su objetivo principal es vigilar el estado de servidores y servicios, avisando a los equipos de TI cuando algo deja de funcionar o baja su rendimiento. Se destaca por su flexibilidad y capacidad de personalización, lo que permite adaptarse a distintos entornos. Sin embargo, esa misma flexibilidad genera complejidad: en organizaciones grandes resulta difícil de administrar y necesita complementarse con interfaces adicionales para mejorar la visualización de la información.
- **Zabbix [7]** : Presentado en el estudio como una solución moderna de código abierto, su propósito es entregar una visión completa y en tiempo real del estado de la infraestructura, mediante paneles gráficos, reportes y mapas dinámicos que facilitan la comprensión del rendimiento de los sistemas. Su fortaleza radica en que reduce el trabajo manual gracias a funciones de detección automática de dispositivos conectados a la red. Sin embargo, una de sus principales debilidades es que, si no se ajustan correctamente los parámetros de alerta, puede generar sobrecarga de notificaciones y alertas falsas, lo que distrae a los equipos de TI y dificulta priorizar los incidentes realmente críticos.
- **ScienceLogic SL1 (SL1) [8]** : Considerado un software de nivel empresarial, su función es ofrecer una visión unificada del estado de los servicios de la organización, mostrando no solo la falla de un equipo, sino también el impacto de dicha falla en toda la operación. Incorpora inteligencia artificial para filtrar alertas repetitivas y priorizar las más críticas, lo que ayuda a los equipos a enfocarse en lo importante. Su debilidad radica en su despliegue, el cual puede resultar complejo, ya que requiere adaptar múltiples integraciones y configurar dependencias entre servicios. Esto implica más tiempo y recursos en la fase inicial antes de obtener beneficios tangibles.
- **NetBox [9]** : Por último se encuentra Netbox, cuyo proposito es actuar como un repositorio central de datos de red, permitiendo organizar y documentar direcciones IP, dispositivos, conexiones físicas y lógicas. Se destaca por ofrecer una plataforma clara para la gestión de inventarios de red y para planificar la infraestructura de manera ordenada. Sin embargo, presenta algunas desventajas: requiere conocimientos técnicos para su configuración inicial, carece de funciones de automatización directa (por lo que debe integrarse con otras herramientas para ejecutar cambios en la red) y no resulta tan intuitivo para usuarios sin experiencia previa.

Tras el análisis de estas herramientas, podemos concluir que la gestión de infraestructura tecnológica constituye un desafío creciente para las empresas, debido a la complejidad de las redes actuales y la criticidad de los servicios que dependen de ellas. Como se evidenció en la revisión, soluciones como **Nagios**, **Zabbix** y **ScienceLogic SL1** aportan capacidades avanzadas de monitoreo y analítica, mientras que plataformas como **NetBox** cumplen un rol fundamental en la documentación e inventario de activos. No obstante, estas alternativas presentan limitaciones significativas al ser evaluadas en conjunto: Nagios y Zabbix requieren un alto grado de configuración manual y conocimientos especializados, lo que eleva su curva de aprendizaje. SL1, si bien ofrece funciones empresariales avanzadas, implica altos costos y dependencia del proveedor. Por último, NetBox carece de capacidades de automatización y monitoreo en tiempo real.

1.4. Justificación de la Propuesta

Tras revisar las principales herramientas de monitoreo y gestión de infraestructura se identifica un patrón común, ninguna ofrece por sí sola una solución integral que combine visibilidad completa, simplicidad operativa y automatización accesible. Mientras unas requieren configuraciones complejas y altos niveles de especialización, otras presentan costos elevados o carecen de capacidades esenciales como monitoreo en tiempo real o integración nativa de procesos.

En consecuencia, las organizaciones terminan dependiendo de la integración de múltiples herramientas para lograr visibilidad completa, lo que incrementa la fragmentación, la redundancia de alertas y la dificultad en la administración. A esto se suma que, en la mayoría de los casos, la visualización de los estados de red no es suficientemente intuitiva para apoyar la toma de decisiones rápida, generando sobrecarga informativa y tiempos de respuesta prolongados.

El presente proyecto se justifica en la necesidad de ofrecer una alternativa que supere estas limitaciones mediante una propuesta que combine, en un mismo entorno, un esquema de visualización gráfica de tipo nodal y mecanismos de automatización accesibles. La representación visual permitirá a los equipos de TI interpretar el estado de la infraestructura de manera clara y contextual, reduciendo la fatiga de alertas y mejorando la capacidad de diagnóstico. La automatización modular, por su parte, facilitará la definición de respuestas a eventos críticos, como notificaciones inmediatas a canales de comunicación corporativos, sin requerir conocimientos técnicos avanzados ni despliegues complejos.

De esta manera, la propuesta se diferencia de las soluciones tradicionales al priorizar la simplicidad de uso y la integración nativa de dos funciones clave: visibilidad integral y capacidad de automatizar tareas repetitivas. Ello no solo mejora la eficiencia operativa y disminuye los tiempos de recuperación, sino que también aumenta la accesibilidad de la herramienta a empresas que necesitan soluciones efectivas sin incurrir en la complejidad o los costos de plataformas más avanzadas.

Por todo lo anterior, se propone en este Proyecto de Título el diseño y validación de una plataforma web de administración visual de infraestructura TI, de nombre **"Nocturne Scope"**, que integre visualización gráfica de nodos, monitoreo en tiempo real y automatización modular de tareas. Esta solución busca consolidar funciones dispersas en un único panel de control, otorgando trazabilidad, facilidad de uso y un modelo adaptable a empresas de distintos tamaños, fortaleciendo así la continuidad operativa y la resiliencia de sus servicios tecnológicos.

1.5. Objetivos del proyecto

1.5.1. Objetivo General

Crear una solución centralizada para la administración de infraestructura TI que unifique en una sola plataforma la supervisión de dispositivos y servicios de red, la visualización gráfica de topología, el registro de eventos y la automatización de tareas operativas, con el propósito de mejorar la eficiencia, reducir los tiempos de respuesta ante incidentes y aumentar la continuidad operativa de los servicios tecnológicos en las organizaciones.

1.5.2. Objetivos Específicos

- (a) Analizar el estado del arte en herramientas de monitoreo, visualización y automatización de infraestructura TI, identificando las limitaciones presentes en soluciones tradicionales y estableciendo los requisitos funcionales y no funcionales para una plataforma unificada.
- (b) Identificar los datos y variables críticas de los dispositivos conectados a la red, para la elaboración de métricas de rendimiento y salud del sistema que permitan evaluar el estado operativo, anticipar fallas y apoyar la toma de decisiones en la gestión de la infraestructura tecnológica.
- (c) Aplicar tecnologías informáticas que faciliten la integración de servicios y herramientas de red en una plataforma unificada, orientada a simplificar el monitoreo, administración y automatización de los sistemas de infraestructura.
- (d) Evaluar la efectividad de la plataforma propuesta mediante pruebas locales y remotas en entornos de red controlados, verificando la precisión del monitoreo, la estabilidad de la comunicación entre dispositivos y la capacidad de respuesta del sistema ante eventos críticos.

1.6. Composición del Informe

El informe se organiza en capítulos que abordan de manera progresiva los elementos clave del proyecto.

En el **Capítulo 1** se presentan el área de estudio, los procesos involucrados y la problemática que motiva el desarrollo de la solución, junto con los objetivos generales y específicos.

El **Capítulo 2** describe el proyecto en detalle, incluyendo los principios de diseño, los objetivos del software, la metodología de desarrollo adoptada, las tecnologías empleadas y el análisis de factibilidad.

El **Capítulo 3** desarrolla la especificación de requerimientos, donde se establecen los límites del sistema, las restricciones técnicas, los requisitos funcionales y no funcionales, así como las interfaces de entrada y salida.

El **Capítulo 4** aborda el análisis del sistema mediante la definición de actores, historias de usuario y casos de uso, apoyados por sus correspondientes diagramas y descripciones.

El **Capítulo 5** expone el diseño del sistema, abarcando los servicios web necesarios, el modelo de datos y los elementos visuales asociados a la interfaz y la navegación. En el sexto capítulo se detalla el proceso de desarrollo, describiendo la arquitectura, la estructura del código y los componentes implementados.

El **Capítulo 6** detalla el proceso de desarrollo, describiendo la arquitectura, la estructura del código y los componentes implementados.

El **Capítulo 7** presenta las estrategias de implantación, junto con el plan de capacitación destinado a facilitar la adopción del sistema.

Por último, **Capítulo 8** presenta las conclusiones obtenidas y propone posibles líneas de trabajo futuro, cerrando así el ciclo completo del proyecto.

Capítulo 2

Proyecto

El presente capítulo tiene por objetivo describir los principales conceptos asociados a la formación de complejos proteicos y a su predicción, facilitando al lector la comprensión de las secciones posteriores.

2.1. Objetivos del Software

2.1.1. Objetivo General

Registrar y monitorear infraestructura de red en tiempo real mediante una visualización interactiva de la topología de la red, permitiendo detectar eventos relevantes, generar alertas oportunas y facilitar la administración de los dispositivos de la empresa.

2.1.2. Objetivos Específicos

- (a) Generar un nodo en la topología de red, que represente dispositivos y servicios, mostrando métricas generales de uso (CPU, GPU, temperatura, etc.), y que permita comprender de manera clara el estado de la infraestructura y sus dependencias.
- (b) Generar alertas con los sucesos importantes ocurridos dentro de la red, con el fin de facilitar el monitoreo histórico, la trazabilidad de incidentes y el análisis de tendencias para la toma de decisiones.
- (c) Integrar un agente para servidores y estaciones de trabajo capaz de recolectar métricas de uso y estado, enviar información de manera segura hacia la plataforma y ejecutar tareas básicas de gestión, manteniendo la confiabilidad y continuidad del monitoreo.

- (d) Automatizar tareas basado en series de acciones configurables, que tendrán efecto sobre los dispositivos conectados en red, como el envío de notificaciones de eventos detectados en la infraestructura, asegurando flexibilidad y facilidad de uso.

2.2. Metodología de Desarrollo

La metodología seleccionada para este proyecto corresponde al **enfoque incremental** [10], debido a que la naturaleza del sistema exige validar de manera continua los avances parciales y asegurar la correcta integración entre sus componentes principales. La plataforma propuesta combina monitoreo en tiempo real, visualización gráfica de topología, generación de reportes y mecanismos de automatización, funciones que deben ser desarrolladas y evaluadas progresivamente para garantizar su estabilidad operativa.

El enfoque incremental permite construir cada módulo de forma independiente, integrándolos luego en un entorno funcional sin comprometer el funcionamiento del resto del sistema, como se representa en la Figura 2.1. Asimismo, el desarrollo de este software requiere interacción constante con dispositivos reales y servicios desplegados en entornos locales o remotos, lo que hace necesario incorporar retroalimentación temprana de las pruebas. La capacidad del método incremental para entregar versiones funcionales en cada etapa facilita la detección oportuna de problemas, reduce riesgos técnicos y mejora la fiabilidad del sistema antes de avanzar al siguiente conjunto de funcionalidades. Dado que la plataforma integrará diversas tecnologías, como Go, Next.js, PostgreSQL, InfluxDB y Docker, este enfoque permite validar la interoperabilidad entre componentes y ajustar progresivamente la arquitectura sin retrasar el desarrollo global.

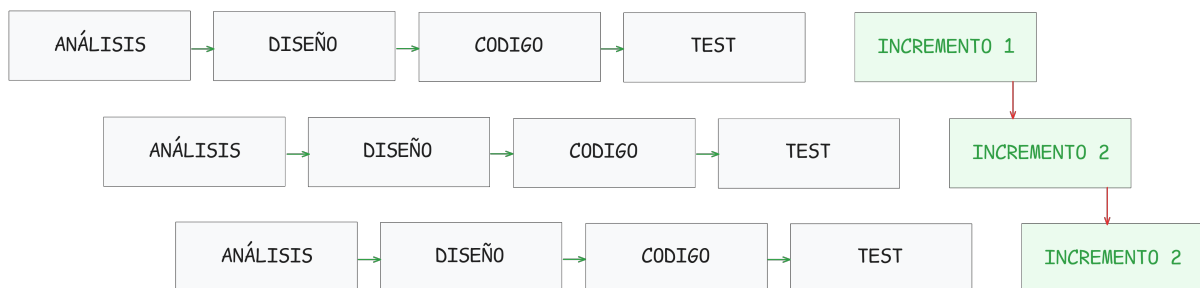


Figura 2.1: Diagrama Metodología Evolutiva

2.3. Tecnologías Utilizadas (Backend, Frontend, Base de Datos)

Durante el desarrollo del sistema propuesto se utilizaran herramientas y tecnologías ampliamente adoptadas en la industria, tanto por su madurez como por su eficiencia en sistemas distribuidos y modernos basados en microservicios. A continuación se detallan las herramientas utilizadas y su propósito dentro del proyecto:

- (a) **Node.js (versión 24.2.0)**: Entorno de ejecución requerido por *Next.js* para compilar y servir la aplicación frontend.
- (b) **Go**: Lenguaje de programación compilado y concurrente, seleccionado por su eficiencia en la ejecución, bajo consumo de recursos y facilidad para desarrollar servicios escalables y de alto rendimiento.
- (c) **Fiber**: Framework web para Go, adoptado para el desarrollo del backend por su simplicidad, velocidad y soporte nativo para middleware y rutas optimizadas.
- (d) **Next.js**: Framework basado en React utilizado para el desarrollo del panel de visualización web, con soporte SSR y SSG, facilitando una interfaz moderna y eficiente para el monitoreo remoto del sistema.
- (e) **InfluxDB**: Base de datos orientada a series temporales, utilizada para almacenar y consultar datos con marcas de tiempo de forma eficiente. Elegida por su alto rendimiento en inserciones, capacidad de agregación temporal y compatibilidad con protocolos como Flux y InfluxQL.
- (f) **PostgreSQL**: Sistema de gestión de base de datos relacional utilizado para almacenar la información histórica de ataques, configuración del sistema y registros de eventos.
- (g) **Docker**: Plataforma de virtualización utilizada para desplegar los honeypots, servicios de análisis y la infraestructura general del sistema en un entorno controlado, modular y replicable.

2.4. Estándares de Documentación

Para asegurar la claridad, trazabilidad y coherencia del desarrollo del software, la documentación del proyecto seguirá los lineamientos definidos en dos normas reconocidas internacionalmente:

- (a) IEEE Std 830-1998 (IEEE Recommended Practice for Software Requirements Specifications), para la elaboración de la Especificación de Requerimientos del Software (SRS). Este estándar define la estructura y contenido de los requerimientos funcionales y no funcionales del sistema, permitiendo una base sólida para el diseño, implementación y validación.
- (b) IEEE Std 829-1998 (IEEE Standard for Software Test Documentation), para la planificación y documentación de las pruebas de software, incluyendo casos de prueba, criterios de aceptación, resultados y análisis de errores. Esto permitirá evaluar objetivamente la calidad del sistema.

Toda la documentación técnica se almacena y versiona en GitHub, facilitando la colaboración, control de versiones y auditoría de cambios. Teniendo esto en cuenta, podemos decir que este enfoque normativo garantizará un marco estructurado que facilita la comprensión entre los distintos actores del proyecto, la futura mantención del sistema, y la correcta validación de sus funcionalidades antes de su entrega final.

2.5. Técnicas y notaciones

Durante el desarrollo del proyecto se utilizaron diversas técnicas y notaciones formales para representar la funcionalidad, arquitectura y estructura del sistema. Estas herramientas permiten modelar de forma precisa y estandarizada los distintos componentes del software, facilitando la comunicación entre los actores del proyecto y asegurando la trazabilidad de los requisitos.

- (a) **Diagramas de Casos de Uso** (UML): Se emplearán para representar gráficamente las interacciones entre los distintos tipos de usuarios y el sistema, permitiendo identificar funcionalidades clave y delimitar el alcance del software.
- (b) **Diagramas BPMN** (Business Process Model and Notation): Utilizados para modelar procesos de negocio relacionados con la gestión de alertas y eventos de seguridad dentro del sistema. Esta notación permitió visualizar claramente los flujos de tareas y decisiones.
- (c) **Modelo Entidad-Relación** (MER) y **Modelo Relacional** (MR): Aplicados en el diseño de la base de datos del sistema, describiendo las entidades relevantes, sus atributos y relaciones, así como la normalización de las tablas.

- (d) **Clean Architecture:** El sistema incorpora los principios de Clean Architecture propuestos por Robert C. Martin [11], con el fin de asegurar facilitar la mantenibilidad y permitir la evolución futura del sistema sin afectar la lógica central. Este enfoque garantiza que las reglas de negocio permanezcan aisladas de detalles externos como frameworks, bases de datos o servicios externos, promoviendo una arquitectura flexible, testeable y preparada para cambios tecnológicos.

2.6. Factibilidad del Proyecto

2.6.1. Factibilidad Técnica

Para determinar la factibilidad técnica del proyecto, se consideran los siguientes factores:

Se cuenta con personal capacitado para el desarrollo del sistema, con conocimientos avanzados en backend, redes, contenedorización, bases de datos y despliegue de servicios. El estudiante posee experiencia práctica en estas áreas, lo que garantiza la capacidad técnica para construir e implementar el sistema completo.

En cuanto al software necesario tanto para el desarrollo como para la explotación del sistema, se utilizan herramientas de código abierto ampliamente adoptadas por la industria, como FastAPI, Docker, PostgreSQL, Next.js y Git. Todas estas herramientas son gratuitas, cuentan con documentación extensa y poseen comunidades activas, lo cual facilita el aprendizaje y la resolución de problemas técnicos durante el ciclo de vida del sistema.

Respecto al hardware, se dispone de un equipo personal que cumple con los requisitos técnicos mínimos para ejecutar los entornos de desarrollo. Además, el sistema será desplegado en un servidor físico de tipo homelab, adquirido a bajo costo, que proporciona capacidad de cómputo suficiente para alojar los servicios de análisis, monitoreo y redirección de ataques mediante contenedores Docker.

REQUERIMIENTOS	DESCRIPCIÓN
Procesador	Intel Core i3 o superior
Memoria RAM	8 GB
Espacio de almacenamiento	10 GB disponibles (HDD o SSD)
Pantalla	Resolución mínima 1280 x 768 px
Conectividad	Acceso a internet estable

Tabla 2.1: Especificación de Hardware requerido en desarrollo del proyecto

REQUERIMIENTOS	DESCRIPCIÓN
Procesador	Intel Xeon E5-2680 v3 (24 hilos @ 3.3GHz)
Memoria RAM	16 GB DDR4
Espacio de almacenamiento	SSD con al menos 100 GB disponibles para contenedores

Tabla 2.2: Especificación de Hardware Servidor requerido en desarrollo del proyecto

Dado que el proyecto cuenta con personal técnicamente capacitado, acceso completo a herramientas de desarrollo de libre distribución, y hardware disponible tanto para el desarrollo como para el despliegue en entorno controlado, se concluye que el proyecto es **técnicamente factible**.

2.6.2. Factibilidad Operativa

El proyecto "Nocturne Scope" va dirigido a un segmento específico de clientes, el cual es profesionales en etapa de integración al área TI con disposición al aprendizaje. Desde la perspectiva estratégica, esperamos que los clientes y jefaturas reconozcan el valor del software dado que permite mitigar los riesgos asociados a los procesos manuales, los cuales representan una fuente significativa de errores y retrasos en la respuesta ante incidentes. La adopción de esta herramienta debe ser vista como un beneficio directo para la continuidad operativa, al facilitar la supervisión de servicios críticos sin depender exclusivamente de personal con experiencia avanzada para la detección de fallas.

Por parte de los usuarios, específicamente el Equipo TI con competencias intermedias, esperamos que exista una alta receptividad hacia la solución debido a que reduce drásticamente la complejidad habitual de las herramientas de monitoreo tradicionales como Nagios o Zabbix. Al ofrecer una visualización gráfica de la topología en formato de nodos y una interfaz intuitiva, el sistema permite que estos nuevos profesionales comprendan rápidamente el estado y las dependencias de la red, superando la barrera de entrada que suponen las configuraciones complejas por línea de comandos.

Finalmente, el proyecto es operativamente viable dado que el segmento al que va dirigido, posee las competencias digitales básicas requeridas para operar plataformas web y gestionar sistemas operativos Linux, habilidades que se complementan perfectamente con el diseño del software.

2.6.3. Factibilidad Económica

Software	Licencia	Costo Licencia
Firefox / Chrome / Brave	MPL 2.0 / BSD / MPL 2.0	Gratuito
Git	GPLv2	Gratuito
GitHub	Propietaria (Plan Free disponible)	Gratuito
Go	BSD-3-Clause	Gratuito
Fiber	MIT	Gratuito
Next.js	MIT	Gratuito
Node.js	MIT	Gratuito
PostgreSQL	PostgreSQL License	Gratuito
InfluxDB v3.4.0	MIT	Gratuito
Docker	Apache 2.0	Gratuito
Docker Compose	Apache 2.0	Gratuito

Tabla 2.3: Licencias del software utilizado en el desarrollo del proyecto

Item	\$ mensual aprox. en CLP	\$ anual aprox. en CLP
Dominio (nic.cl)	–	\$9.900 CLP
Hosting VPS (HOSTINGER)	\$6.594	\$96.113 CLP

Tabla 2.4: Costo hosting y dominio. Precios obtenidos de NIC¹ y Hostinger²

Recurso Humano	Cantidad personal	Sueldo aprox. en CLP por mes	Sueldo aprox. en CLP por duración proyecto (x meses)
Desarrollador Full-Stack	1 persona	\$1.305.000	\$3.915.000 (3 meses)

Tabla 2.5: Cálculo de costo de desarrollo y soporte basado en sueldo Estudiante³ y Desarrollador Junior⁴.

¹Costo de Dominio ".CL": <https://www.nic.cl/dominios/tarifas.html>

²Costo de Servidor VPS privado: <https://www.hostinger.com/vps-hosting>

³Sueldo Estudiante: <https://www.dep.usach.cl/es/ayudantias>

⁴Sueldo Desarrollador Full-Stack Junior: <https://www.freelancermap.com/blog/es/freelance-chile/>

Flujo de caja

Para asegurar la viabilidad económica del proyecto, se empleará el indicador del Valor Actual Neto (VAN) como medida. Para ello, se realizará el cálculo del flujo de caja correspondiente a la inversión inicial, así como se proyectarán los flujos de caja para los primeros 5 años. Estos datos se presentan en detalle en la siguiente tabla:

Recurso Humanos	Año 0	Año 1	Año 2	Año 3	Año 4	Año 5
(+) Ingresos						
Beneficios		\$1.395.000	\$7.380.000	\$16.020.000	\$24.660.000	\$33.300.000
(-) Costos						
Servicios	-	\$337.500	\$361.500	\$483.000	\$507.000	\$628.500
Soporte y Mantención	\$3.915.000	\$950.000	\$950.000	\$950.000	\$1.425.000	\$1.425.000
TOTAL	\$3.915.000	\$1.287.500	\$1.311.500	\$1.433.000	\$1.932.000	\$2.053.500

Tabla 2.6: Calculo costo de desarrollo y soporte.

Justificación de Flujos Proyectados: Los ingresos operacionales “Beneficios” se fundamentan en un modelo de suscripción al programa de \$15.000 CLP mensuales, proyectando una cartera inicial de 15 clientes y un crecimiento conservador de 4 nuevas suscripciones por mes. Por el lado de los egresos, los costos operacionales reflejan la escalabilidad técnica y comercial: el ítem “Servicios” aumenta progresivamente debido a la migración escalonada de planes de infraestructura VPS de Hostinger⁵ y una mayor inversión en marketing. Finalmente, el costo de “Soporte y Mantención” se basa en una estimación de horas-hombre valorizadas según tarifas de mercado⁶ para perfiles TI, contemplando un incremento estratégico en los años 4 y 5 para asegurar la calidad del servicio ante el aumento sustancial de la base instalada.

⁵Ver planes en: <https://www.hostinger.cl/hosting-vps>

⁶Sueldo Desarrollador Full-Stack Junior: <https://www.freelancemap.com/blog/es/freelance-chile/>

Cálculo del V.A.N

A continuación, se calculará el VAN con una tasa de descuento del 10

Año	Flujo de Caja
Año 0	$-3,915,000/(1 + 0,10)^0 = -3,915,000$
Año 1	$107,500/(1 + 0,10)^1 = 97,727$
Año 2	$6,068,500/(1 + 0,10)^2 = 5,015,289$
Año 3	$14,587,000/(1 + 0,10)^3 = 10,959,429$
Año 4	$22,728,000/(1 + 0,10)^4 = 15,523,530$
Año 5	$31,246,500/(1 + 0,10)^5 = 19,401,618$
VAN TOTAL $\approx$ \$47.082.593	

Tabla 2.7: Cálculo del VAN con tasa del 10 %

En este caso, el VAN obtenido es positivo \$ **47.082.593**, lo que indica que el proyecto es viable desde una perspectiva económica.

2.6.4. Conclusión de Factibilidad

Gracias al análisis realizado en los puntos anteriores, se puede concluir que el proyecto es viable y altamente rentable desde una perspectiva económica y financiera. El principal indicador que respalda esta afirmación es el Valor Actual Neto (VAN), cuyo cálculo arroja un resultado positivo de \$47.082.593. Al obtener una cifra superior a cero, se confirma matemáticamente que la iniciativa no solo tiene la capacidad de recuperar la inversión inicial de \$3.915.000 y cubrir la tasa de costo de oportunidad del 10 %, sino que además genera una creación de valor sustancial para los inversionistas en términos de valor presente.

Capítulo 3

Especificación de Requerimientos - Producto de Software

El presente capítulo tiene por objetivo describir de manera detallada los requerimientos del producto de software, incluyendo sus funcionalidades principales, las interfaces externas de entrada y salida, así como los límites operativos del sistema.

3.1. Límites

Esta primera versión de la plataforma se enfoca en la visualización gráfica de la topología de red, el monitoreo de métricas generales de los dispositivos conectados, principalmente servidores y estaciones de trabajo, la recopilación de eventos relevantes y la automatización de tareas básicas mediante flujos configurables. El sistema permitirá el envío de notificaciones automáticas por correo electrónico y mensajería instantánea, y mostrará en tiempo real el estado operativo de los equipos que cuenten con el agente instalado.

No obstante, el alcance de esta versión no abarca la integración de routers, switches u otros dispositivos de red, priorizando la estabilidad del sistema y la validación del modelo visual y funcional sobre infraestructura de servidores y estaciones de trabajo. La automatización se limitará a acciones predefinidas, como el envío de alertas o la ejecución de tareas locales, sin integración con sistemas externos de gestión de incidentes. Finalmente, la administración simultánea de múltiples redes o sedes no estará contemplada en esta etapa, privilegiando un entorno controlado que permita asegurar la confiabilidad de la base tecnológica de la propuesta.

3.2. Restricciones Técnicas

La implementación de la presente plataforma implica ciertas limitaciones técnicas asociadas a su alcance inicial y a las características del entorno de infraestructura donde se desplegará. En esta primera etapa, el sistema se encuentra orientado al monitoreo y administración de servidores y estaciones de trabajo con sistema operativo Linux, sin incluir aún la integración con dispositivos de red como routers o switches, cuya incorporación requeriría configuraciones adicionales y protocolos específicos. Asimismo, la recopilación de métricas dependerá de la instalación y correcto funcionamiento de un agente en cada equipo, lo que podría representar un esfuerzo adicional de configuración y mantenimiento para el área de TI.

Del mismo modo, la diversidad de distribuciones y versiones de Linux dentro de una organización puede generar diferencias en la disponibilidad o precisión de ciertos indicadores, particularmente en lo relacionado con sensores térmicos, uso de GPU o recursos físicos. En redes con un número elevado de equipos, la visualización gráfica de la topología podría experimentar una disminución en el rendimiento, afectando la fluidez del monitoreo en tiempo real. Además, la capacidad de almacenamiento de datos históricos será limitada, restringiendo la trazabilidad de métricas a un período acotado para evitar sobrecarga en el sistema.

3.3. Requerimientos Funcionales

id	El sistema permite
RF_01	El Encargado TI y el Equipo TI podrán autenticarse mediante el uso de un correo electrónico y una contraseña válidos. Antes del inicio de sesión se valida que el correo exista y que las credenciales coincidan con los registros almacenados en la base de datos. En caso de credenciales incorrectas o usuario inactivo, se mostrará un mensaje de error sin revelar detalles sensibles. Una vez autenticado, el sistema genera una sesión con vigencia de un día.
RF_02	El Encargado TI podrá generar y eliminar tokens de API personales para conectar sus dispositivos al sistema de monitoreo remoto. Cada token se asocia al usuario que lo creó, cuenta con permisos limitados y una fecha de expiración configurable. El token se muestra solo una vez al momento de su creación y se almacena cifrado en la base de datos. Solo el propietario o un administrador podrán gestionarlo.
RF_03	El Encargado TI y el Equipo TI podrán conectar dispositivos al sistema de monitoreo. Al conectarse por primera vez, se desplegará una interfaz básica donde deberán registrar la información del dispositivo (nombre, token, tipo de dispositivo, etc.). El backend verificará si el dispositivo ya está registrado; en caso afirmativo, actualizará su registro; en caso contrario, lo creará en la base de datos. El dispositivo quedará inmediatamente asociado al usuario propietario del token utilizado.
RF_04	El Encargado TI y el Equipo TI podrán visualizar en un <i>dashboard</i> interactivo las métricas de rendimiento de los dispositivos conectados, como uso de CPU, memoria RAM, almacenamiento, red, temperatura, sistema operativo y tiempo de actividad. La visualización se realizará en intervalos de tiempo configurables (10 segundos por defecto). Los usuarios solo podrán ver las métricas de sus dispositivos asociados, mientras que los encargados de TI podrán visualizar todos los dispositivos del sistema.

id	El sistema permite
RF_05	El Encargado TI y el Equipo TI podrán generar reportes con la información histórica de los dispositivos conectados al sistema. Para ello deberán especificar un rango temporal de inicio y fin, así como seleccionar los dispositivos que desean incluir en el informe. El reporte incluirá encabezados con la fecha de generación, el autor (encargado de TI solicitante) y los filtros aplicados (rango temporal y lista de dispositivos). Una vez generado, el reporte quedará inmediatamente disponible para su descarga segura.
RF_06	El Encargado TI y el Equipo TI podrán visualizar la topología de los dispositivos que administran en una interfaz gráfica, con la capacidad de ordenar la vista en forma de elementos interconectados similares a un diagrama de flujo de nodos. Cualquier personalización o reordenamiento que realice el encargado de TI solo se reflejará y persistirá en sus propias sesiones. Tras una visualización exitosa, la interacción con cualquiera de los nodos desplegará la información detallada del dispositivo y las acciones de gestión disponibles, facilitando la administración visual de la infraestructura.
RF_07	El Encargado TI y el Equipo TI tendrán la capacidad de automatizar tareas de monitoreo, basándose en condiciones lógicas como el cumplimiento de una flag o que una métrica supere un umbral. Las acciones de automatización a realizar incluyen el envío de correos de notificación, el apagado remoto de un equipo, entre otras. Las tareas de automatización se ejecutarán automáticamente e inmediatamente cuando se cumpla la condición o <i>trigger</i> definido en el flujo. Las automatizaciones que involucren acciones críticas para el sistema, como el apagado de un equipo, requerirán la confirmación mediante el ingreso de la contraseña del administrador, por motivos de seguridad. Después de la ejecución de cualquier tarea de automatización, exitosa o fallida, el sistema registrará el evento, el resultado de la acción y el autor de la configuración en un log de auditoría para su posterior seguimiento.

id	El sistema permite
RF_08	El Encargado TI y el Equipo TI tendrán la capacidad de generar informes de monitoreo a partir de los datos recopilados por el sistema, seleccionando el periodo de tiempo, los dispositivos a incluir y las métricas relevantes, tales como disponibilidad, uso de CPU, memoria, red y eventos registrados. Los informes podrán exportarse en formato PDF y contendrán gráficos, tablas y un resumen ejecutivo del estado de la infraestructura. El sistema permitirá incluir comentarios adicionales por parte del Encargado TI antes de su generación. Una vez generado el informe, el sistema almacenará una copia en el historial de informes, junto con la fecha de creación y el usuario que lo generó, para efectos de auditoría y trazabilidad.

Tabla 3.2: Requerimientos Funcionales

3.4. Requerimientos No Funcionales

id	El sistema
RNF_01	El sistema debe garantizar una alta capacidad de respuesta para asegurar la eficiencia del encargado de TI, especialmente al momento de consultar información crítica. Por ello, la interfaz de visualización de la topología gráfica y el dashboard de métricas deben cargar y actualizarse en un tiempo no mayor a 5 segundos, excluyendo la latencia propia de la red del usuario. Este límite es vital para cumplir con el objetivo de facilitar la rápida comprensión del estado de la red.
RNF_02	Se requiere almacenar grandes volúmenes de datos de series temporales de forma eficiente, es por esto que la base de datos seleccionada debe demostrar escalabilidad para manejar la carga operativa. Se establece que el sistema debe soportar la inserción continua de datos de monitoreo generados por al menos 10 dispositivos simultáneos sin que esto provoque una degradación en el rendimiento de la plataforma. Este requisito garantiza la capacidad del sistema para crecer y ser viable en entornos empresariales medianos, manteniendo un alto rendimiento en las inserciones y consultas de agregación temporal.
RNF_03	Considerando que el sistema recolectará y gestionará información sensible de la infraestructura, se requiere seguridad en la comunicación. La transmisión de datos, incluyendo métricas de uso y comandos de gestión, entre los dispositivos y el servidor backend de la plataforma debe realizarse exclusivamente a través de protocolos de comunicación cifrados (ejem. TLS/SSL). Esto es fundamental para garantizar la integridad y confidencialidad de la información.

RNF_04	El diseño de la interfaz de usuario debe priorizar la sencillez, claridad y la intuición para que el encargado de TI pueda interpretar el estado operativo de la infraestructura de manera clara y contextual. La organización visual, incluyendo la paleta de colores y la tipografía, deberá ser coherente en todas las pantallas para reducir la carga cognitiva, permitiendo así una toma de decisiones más rápida y mejorando la eficiencia operativa general.
RNF_05	El sistema de monitoreo debe ser robusto y tolerante a fallos del entorno. Se requiere que la plataforma sea capaz de gestionar y tolerar la caída o desconexión temporal de uno o varios dispositivos sin que esto comprometa la funcionalidad del dashboard principal, ni detenga la recolección o procesamiento de métricas provenientes del resto de los equipos.

Tabla 3.4: Requerimientos No Funcionales

3.5. Interfaces externas de Entrada

ID	Nombre del ítem	Detalle de Datos contenidos en ítem
DE_01	Datos de Usuarios	NOMBRE, CORREO, CONTRASEÑA , ROL.
DE_02	Datos de Token	TOKEN_NOMBRE, FECHA_EXPIRACION, PERMISOS_ASIGNADOS
DE_03	Datos de Dispositivo	NOMBRE_DISPOSITIVO, TOKEN_ASOCIACION, TIPO_DISPOSITIVO, DIRECCION_IP, SISTEMA_OPERATIVO.
DE_04	Datos de Monitoreo	TOKEN_DISPOSITIVO, TIMES-TAMP, USO_CPU, USO_RAM, USO_ALMACENAMIENTO, TRAFICO_RED, TEMPERATURA, TIEMPO_ACTIVIDAD.

Tabla 3.5: Interfaces de Entrada

3.6. Interfaces externas de Salida

ID	Nombre del ítem	Detalle de Datos contenidos en ítem	Medio Salida
IS_01	Dashboard de Métricas en Tiempo Real	NOMBRE_DISPOSITIVO, ESTADO_OPERATIVO, USO_CPU, USO_RAM, USO_ALMACENAMIENTO, TRAFICO_RED, TEMPERATURA, SISTEMA_OPERATIVO, TIEMPO_ACTIVIDAD.	Pantalla Interactiva (Web)
IS_02	Vista Topología de Red	ID_NODO, NOMBRE_DISPOSITIVO, DIRECCION_IP	Pantalla Interactiva (Web)
IS_03	Reporte de Dispositivos	FECHA_GENERACION, RANGO_TEMPORAL, FILTROS_APLICADOS, DISPOSITIVOS, METRICAS_DISPOSITIVOS.	Archivo PDF/XLS
IS_04	Notificación de Automatización	ASUNTO_ALERTA, CUERPO_MENSAJE, FECHA_EVENTO.	Correo Electrónico, Mensajería Instantánea.

Tabla 3.6: Interfaces de Salida

Capítulo 4

Análisis - Casos de Uso

El presente capítulo describe los casos de uso que estructuran el funcionamiento de del proyecto desde la perspectiva de los actores que interactúan con el sistema. Su objetivo es definir de manera precisa las funcionalidades ofrecidas, las responsabilidades de cada usuario y las relaciones entre las distintas operaciones que componen la plataforma.

4.1. Historias de usuario

id	historia de usuario
HU_01	Como Encargado TI, busco visualizar y eliminar claves para controlar de forma segura el acceso de los dispositivos al sistema de monitoreo.
HU_02	Como Equipo TI, visualizar las métricas de rendimiento de los dispositivos relevantes conectados a mí red, para identificar rápidamente cuellos de botella o fallas operativas (p. ej., uso de CPU, RAM, temperatura).
HU_03	Como Encargado TI o Equipo TI, ver los dispositivos de la red en un esquema gráfico para comprender visualmente las dependencias y el estado operativo de la infraestructura.
HU_04	Como Miembro del Equipo TI, configurar flujos de automatización basados en condiciones lógicas y umbrales de métricas, para implementar respuestas automáticas e inmediatas ante la detección de eventos críticos en la infraestructura.
HU_05	Como Encargado TI, generar reportes históricos para analizar las tendencias de rendimiento y la salud de los dispositivos durante un rango temporal específico.
HU_06	El Equipo TI, necesita poder filtrar y ordenar los dispositivos y métricas por estado operativo (crítico, alerta, normal), para priorizar mi atención en los problemas más urgentes que requieran una intervención inmediata.

Tabla 4.1: Historias de Usuario.

4.2. Actores de casos de uso

Los actores que interactúan con el sistema se detallan en la Tabla 4.2.

Actor	Cargo(s)	Funciones en la empresa	Nivel de conocimientos técnicos requeridos	Nivel privilegio en el sistema
Encargado TI	Ingeniero de Redes, Encargado TI	Responsable de la estrategia, seguridad y gestión de acceso de la infraestructura tecnológica. Supervisa la operación crítica y define políticas de red.	Avanzado. Requiere un entendimiento de seguridad de red, configuración de agentes y protocolos de autenticación.	Acceso Total.
Equipo TI	Ingeniero de Redes, Analista de Soporte	Responsable de la operación, mantenimiento y diagnóstico diario de los dispositivos y servicios de red, asegurando la continuidad operativa.	Intermedio. Necesita conocimiento de redes, automatización, diagnóstico y operación de servicios TI.	Alto.

Tabla 4.2: Actores de casos de usos.

En el caso de que existen generalizaciones de actores incluya los diagramas correspondientes.

4.3. Diagramas y Especificación de casos de uso

4.3.1. Casos de Uso: Dispositivo

CU_01: Generar Token

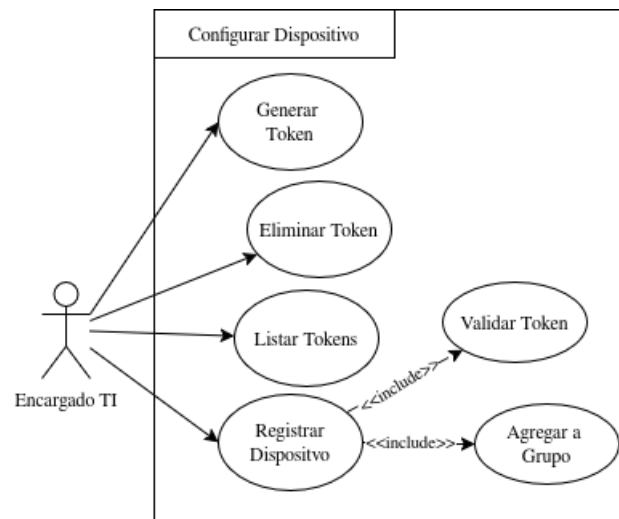


Figura 4.1: Diagrama de Casos de Uso Dispositivo

Precondiciones El usuario debe estar registrado y autenticado en el sistema como Encargado TI. Debe existir al menos un grupo creado por el usuario.

Actor	Aplicación
1) El usuario selecciona la opción “Generar Token”.	1.1) El sistema despliega un formulario que solicita: nombre del token y grupo asociado.
2) El usuario completa los campos requeridos y selecciona el grupo.	2.1) El sistema muestra la lista de grupos disponibles y asociados al usuario.
3) El usuario confirma la creación del token.	3.1) El sistema valida los campos del formulario (formato, obligatoriedad, pertenencia del grupo). 3.2) Si la validación es correcta, el sistema genera un token único y lo asocia al grupo seleccionado y al usuario. 3.3) El sistema almacena en la base de datos los metadatos del token (fecha, grupo, estado, expiración).

Flujo de eventos alternativos	
2.a) El usuario no selecciona un grupo o selecciona un grupo inválido.	2.a.1) El sistema informa que debe seleccionar un grupo válido y mantiene el formulario abierto.
3.b) El usuario supera el límite de tokens permitidos.	3.b.1) El sistema rechaza la operación indicando que se alcanzó el máximo de tokens permitidos.
3.c) El formulario contiene datos inválidos o incompletos.	3.c.1) El sistema resalta los errores y solicita corrección antes de generar el token.

Postcondiciones El sistema crea un token único, activo y asociado a un grupo del usuario. Queda disponible para su uso en el CU_04 Registrar Dispositivo.

CU_04: Registrar Dispositivo

Precondiciones El usuario debe estar registrado y autenticado como Encargado TI. Debe existir un token activo asociado al usuario. El dispositivo debe poder enviar o ingresar el token al sistema.

Actor	Aplicación
1) El usuario inicia el proceso de registrar un dispositivo.	1.1) El agente solicita ingresar el token y, opcionalmente, un nombre para el dispositivo.
2) El usuario introduce el token generado anteriormente.	2.1) El sistema ejecuta el CU_05: Validar Token para comprobar su vigencia, pertenencia y estado.
3) El usuario confirma el registro.	3.1) Si el token es válido, el sistema crea un nuevo registro del dispositivo asociado al usuario. 3.2) El sistema ejecuta el CU_06: Agregar a Grupo para asociar el dispositivo al grupo definido por el token.
4) El usuario recibe la confirmación.	4.1) El sistema confirma el registro del dispositivo y muestra el grupo de pertenencia.
Flujo de eventos alternativos	
2.a) El usuario ingresa un token inválido, expirado o revocado.	2.a.1) El sistema informa que el token no es válido y no registra el dispositivo.
2.b) El token pertenece a otro usuario.	2.b.1) El sistema rechaza el registro e informa que el token no corresponde a la cuenta actual.
3.c) El grupo asociado al token está en su capacidad máxima de dispositivos.	3.c.1) El sistema informa que no es posible agregar más dispositivos al grupo.

3.d) Ocurre un error de comunicación o fallo interno.	3.d.1) El sistema informa el error al usuario y permite reintentar posteriormente.
---	--

Postcondiciones El dispositivo queda registrado y asociado al grupo correspondiente. Comienza a estar disponible en el panel de monitoreo del sistema.

CU_02: Eliminar Token

Descripción Permite al usuario eliminar o revocar un token previamente generado, de modo que no pueda ser utilizado por nuevos dispositivos para registrarse en el sistema.

Precondiciones El usuario debe estar autenticado como Encargado TI y debe existir al menos un token asociado a su cuenta.

CU_03: Listar Tokens

Descripción Permite visualizar el listado de tokens del usuario, mostrando información relevante como estado, fecha de creación, fecha de expiración, grupo asociado y permisos.

Precondiciones El usuario debe estar autenticado como Encargado TI.

CU_05: Validar Token

Descripción Caso de uso utilizado por otros procesos para verificar que un token sea válido, activo, no esté expirado y pertenezca al usuario correspondiente.

Precondiciones El sistema debe recibir un token desde el dispositivo o la interfaz del usuario.

CU_06: Agregar a Grupo

Descripción Caso de uso que asocia un dispositivo previamente validado a un grupo del usuario, siguiendo las reglas definidas para la organización de dispositivos.

Precondiciones El dispositivo debe haber sido identificado y debe existir un grupo válido perteneciente al usuario.

4.3.2. Casos de Uso: Reporte

CU_01: Crear Reporte

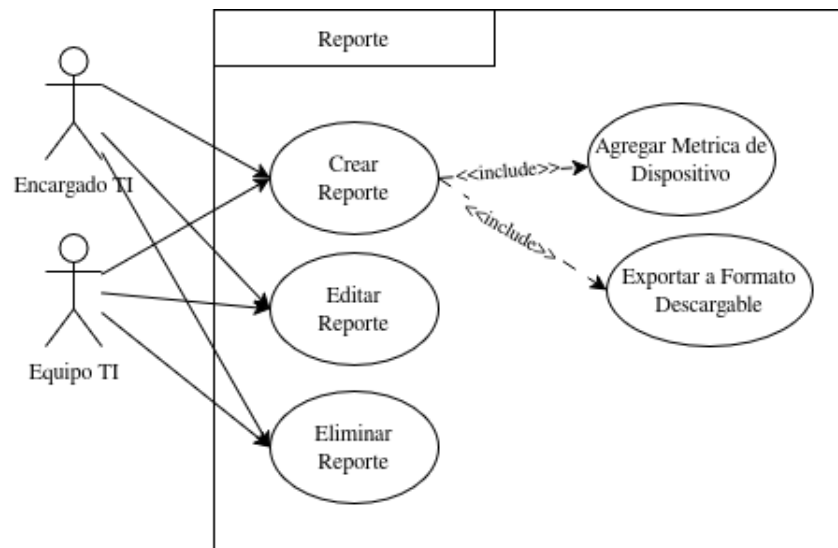


Figura 4.2: Diagrama de Casos de Uso Reporte

Precondiciones El usuario debe estar autenticado. El sistema debe contar con dispositivos registrados y métricas disponibles para incluir en el reporte.

Actor	Aplicación
1) El usuario selecciona la opción “Crear Reporte”.	1.1) El sistema despliega un formulario solicitando: título, periodo de fechas, dispositivos a incluir, métricas seleccionadas, puertos auditados y tráfico auditado.
2) El usuario selecciona el rango de fechas y los dispositivos que desea incluir.	2.1) El sistema muestra la lista de dispositivos disponibles del usuario y valida que exista información en el rango solicitado.
3) El usuario selecciona las métricas, puertos y tráfico que desea agregar al reporte.	3.1) El sistema ejecuta el CU_04: Agregar Métrica de Dispositivo para incorporar las métricas seleccionadas al contenido del reporte.
4) El usuario confirma la creación del reporte.	4.1) El sistema genera un nuevo registro en la base de datos, asignándole un identificador único. 4.2) El sistema compila la información seleccionada para preparar el documento exportable.

5) El usuario selecciona la opción “Generar Informe”.	5.1) El sistema ejecuta el CU_05: Exportar a Formato Descargable, generando un archivo PDF. 5.2) El sistema descarga automáticamente el informe en el navegador del usuario.
Flujo de eventos alternativos	
2.a) No existen datos registrados en el periodo seleccionado.	2.a.1) El sistema informa que no hay información disponible para el rango seleccionado y solicita cambiar el periodo.
3.b) El usuario no selecciona métricas o dispositivos.	3.b.1) El sistema indica que debe seleccionar al menos un dispositivo y una métrica para continuar.
4.c) Ocurre un error interno durante la creación del registro.	4.c.1) El sistema muestra un mensaje de error e invita a reintentar más tarde.

Postcondiciones El reporte queda almacenado en la base de datos, con un identificador único visible en el PDF generado. El usuario puede editarlo o eliminarlo posteriormente.

CU_02: Editar Reporte

Descripción Permite al usuario modificar un reporte previamente creado, actualizando su título, periodo de fechas, dispositivos incluidos, métricas seleccionadas, puertos auditados o tráfico auditado.

Precondiciones El usuario debe estar autenticado y debe existir un reporte previamente creado en el sistema.

CU_03: Eliminar Reporte

Descripción Permite al usuario eliminar un reporte existente, removiéndolo de la base de datos y dejándolo inaccesible para futuras consultas.

Precondiciones El usuario debe estar autenticado y debe existir un reporte registrado en el sistema.

CU_04: Agregar Métrica de Dispositivo

Descripción Caso de uso auxiliar que permite incorporar al reporte métricas seleccionadas por el usuario, tales como CPU, RAM, almacenamiento, temperatura, puertos auditados o tráfico capturado por los dispositivos.

Precondiciones Debe existir un reporte en proceso de creación o edición.

CU_05: Exportar a Formato Descargable

Descripción Genera un archivo PDF descargable con toda la información seleccionada para el reporte, incluyendo su identificador único.

Precondiciones Debe existir un reporte válido con datos completos para exportación.

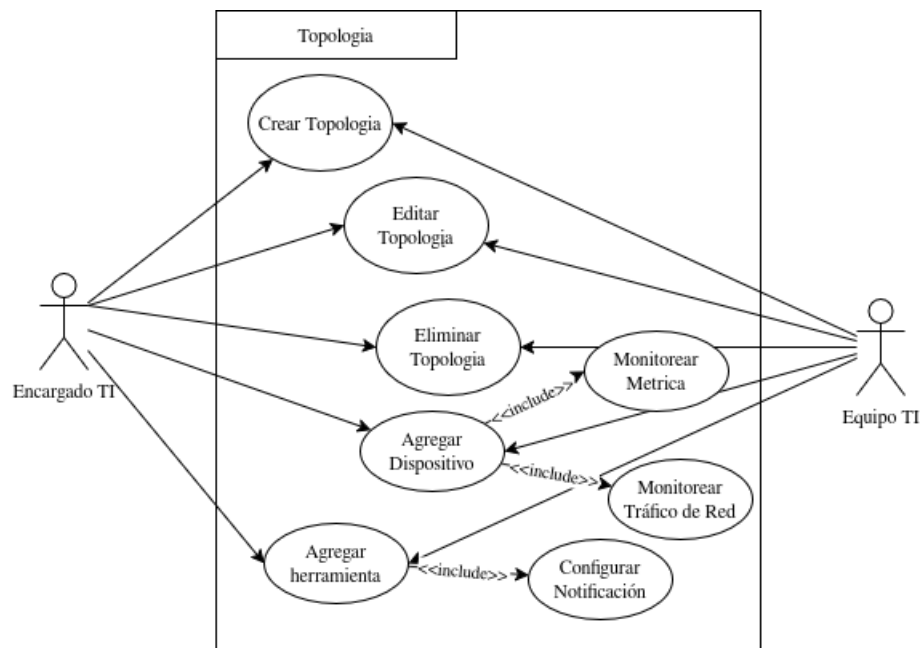
4.3.3. Casos de Uso: Topología

Figura 4.3: Diagrama de Casos de Uso Reporte

CU_01: Crear Topología

Precondiciones El usuario debe estar autenticado en el sistema.

Actor	Aplicación
1) El usuario selecciona la opción “Crear Topología” en el módulo de topologías.	1.1) El sistema muestra un formulario simple solicitando el nombre de la topología .
2) El usuario ingresa el nombre de la nueva topología.	2.1) El sistema valida que el nombre no esté vacío y que no exista otra topología del mismo usuario con el mismo nombre.

3) El usuario confirma la creación de la topología.	3.1) El sistema crea el registro de la topología en la base de datos, asociándolo al usuario. 3.2) El sistema inicializa un lienzo vacío para que el usuario pueda comenzar a agregar dispositivos y herramientas.
4) El usuario accede al lienzo de la topología recién creada.	4.1) El sistema muestra el lienzo vacío correspondiente a la topología, listo para edición.
Flujo de eventos alternativos	
2.a) El usuario no ingresa nombre para la topología.	2.a.1) El sistema indica que el nombre es obligatorio y no permite continuar hasta que se complete el campo.
2.b) El nombre ingresado ya existe para otra topología del mismo usuario.	2.b.1) El sistema informa que ya existe una topología con ese nombre y solicita ingresar uno distinto.
3.c) Se produce un error interno al crear la topología.	3.c.1) El sistema informa el error y no crea el registro, invitando al usuario a reintentar más tarde.

Postcondiciones Se crea una nueva topología asociada al usuario, con su correspondiente canvas vacío disponible para agregar dispositivos y herramientas.

CU_04: Agregar Dispositivo

Precondiciones El usuario debe estar autenticado. Debe existir al menos una topología creada. Deben existir dispositivos registrados previamente en el sistema (por medio del módulo de tokens).

Actor	Aplicación
1) El usuario abre una topología existente y selecciona la opción “Agregar Dispositivo”.	1.1) El sistema despliega un panel o cuadro de diálogo con el listado de dispositivos registrados disponibles para el usuario.
2) El usuario selecciona uno de los dispositivos del listado para incorporarlo a la topología.	2.1) El sistema valida que el dispositivo pertenezca al usuario y que pueda ser agregado a la topología seleccionada.

3) El usuario confirma la adición del dispositivo.	3.1) El sistema agrega un nodo de dispositivo al lienzo de la topología, representando gráficamente al dispositivo. 3.2) El sistema registra en la base de datos la relación entre la topología y el dispositivo. 3.3) El sistema ofrece opciones para asociar monitoreo de estado y tráfico al dispositivo, invocando cuando corresponda el CU_05: Monitorear Métrica y el CU_06: Monitorear Tráfico de Red.
4) El usuario visualiza el dispositivo en el diagrama.	4.1) El sistema muestra el nodo del dispositivo dentro del flujo, permitiendo conectarlo con otras herramientas (por ejemplo, notificaciones o visualización de métricas).
Flujo de eventos alternativos	
2.a) No hay dispositivos registrados para el usuario.	2.a.1) El sistema informa que no existen dispositivos disponibles y sugiere registrar dispositivos antes de agregarlos a la topología.
2.b) El dispositivo ya fue agregado previamente a la misma topología (según la política definida).	2.b.1) El sistema informa que el dispositivo ya se encuentra en la topología y no permite duplicarlo si así se define.
3.c) Ocurre un error al registrar la asociación entre dispositivo y topología.	3.c.1) El sistema informa el error y no agrega el nodo al canvas.

Postcondiciones El dispositivo queda asociado a la topología y representado como un nodo en el canvas. Puede ser conectado a herramientas y participar en flujos de automatización y monitoreo.

CU_07: Agregar Herramienta

Precondiciones El usuario debe estar autenticado. Debe existir al menos una topología creada y abierta en el canvas. Debe existir un catálogo de herramientas disponible (por ejemplo: notificación, disparadores, visualización de métricas, etc.).

Actor	Aplicación
1) El usuario, dentro del canvas de una topología, selecciona la opción “Agregar Herramienta”.	1.1) El sistema muestra un panel o menú lateral con las distintas herramientas disponibles (por ejemplo, herramientas de notificación, de disparo/trigger, de visualización de estadísticas o métricas).
2) El usuario selecciona el tipo de herramienta que desea agregar.	2.1) El sistema solicita los parámetros mínimos de configuración de la herramienta (por ejemplo, tipo de canal de notificación, origen de datos, condiciones de disparo, etc.).
3) El usuario completa la configuración básica de la herramienta y confirma su creación.	3.1) El sistema crea un nodo de herramienta en el lienzo de la topología. 3.2) El sistema registra en la base de datos la herramienta asociada a la topología y su configuración. 3.3) Si la herramienta corresponde a un canal de notificación, el sistema invoca el CU_08: Configurar Notificación para completar los parámetros específicos de notificación.
4) El usuario visualiza la herramienta dentro del flujo de automatización.	4.1) El sistema muestra el nodo de la herramienta listo para ser conectado con dispositivos u otros nodos dentro de la topología.
Flujo de eventos alternativos	
2.a) El usuario no selecciona ningún tipo de herramienta.	2.a.1) El sistema informa que debe seleccionar una herramienta para continuar y mantiene el panel abierto.
3.b) Los parámetros de configuración ingresados son incompletos o inválidos.	3.b.1) El sistema indica los campos con error y no crea la herramienta hasta que se corrijan.
3.c) Se produce un error interno al registrar la herramienta.	3.c.1) El sistema informa el error y no agrega el nodo al canvas.

Postcondiciones La herramienta queda asociada a la topología y representada como un nodo en el canvas, disponible para integrarse en el flujo de automatización y monitoreo.

CU_02: Editar Topología

Descripción Permite modificar una topología existente, incluyendo cambios en su nombre y en la organización de los nodos dentro del canvas (dispositivos, herramientas y conexiones).

Precondiciones El usuario debe estar autenticado y debe existir al menos una topología previamente creada.

CU_03: Eliminar Topología

Descripción Permite eliminar una topología completa, incluyendo sus asociaciones con dispositivos y herramientas dentro del sistema.

Precondiciones El usuario debe estar autenticado y debe existir una topología seleccionada para eliminación.

CU_05: Monitorear Métrica

Descripción Caso de uso auxiliar que permite configurar y habilitar la recolección y visualización de métricas de hardware o estado del dispositivo dentro de una topología (por ejemplo, CPU, memoria, almacenamiento, temperatura).

Precondiciones Debe existir un dispositivo asociado a una topología y una herramienta o nodo que consuma estas métricas.

CU_06: Monitorear Tráfico de Red

Descripción Caso de uso auxiliar que permite habilitar la obtención y análisis de tráfico de red asociado a un dispositivo dentro de la topología (por ejemplo, puertos auditados, volumen de tráfico, patrones de conexión).

Precondiciones Debe existir un dispositivo asociado a una topología con capacidad de exponer información de tráfico de red.

CU_08: Configurar Notificación

Descripción Caso de uso auxiliar que permite definir cómo y cuándo se enviarán notificaciones asociadas a eventos de la topología, configurando canales (correo, mensajería, etc.) y condiciones de disparo.

Precondiciones Debe existir una herramienta de notificación asociada a una topología que requiera configuración de sus parámetros.

Capítulo 5

Diseño

Este capítulo presenta el diseño general del sistema propuesto, abarcando tanto la arquitectura de los servicios como la estructura de datos y la interfaz web. Se describe cómo los componentes fueron organizados para cumplir con los requerimientos funcionales y no funcionales definidos previamente. Además, se detallan los servicios implementados, el modelo de datos relacional y los elementos gráficos esenciales de la interfaz, permitiendo establecer una base clara para su posterior desarrollo e implementación.

5.1. Descripción de los servicios web - necesarios

El funcionamiento de la plataforma se sustenta en una arquitectura de microservicios interconectados que operan de manera coordinada para garantizar el monitoreo y la gestión de la infraestructura. El núcleo lógico del sistema reside en el **Servicio de Backend**, implementado como una API RESTful utilizando el lenguaje de programación Go y el framework Fiber; este componente es el responsable de procesar la lógica de negocio, gestionar la autenticación de usuarios mediante tokens y orquestar el flujo de datos entre los agentes de recolección y la interfaz de usuario. Complementariamente, la capa de presentación es gestionada por el **Servicio de Frontend**, desarrollado en Next.js, el cual entrega la interfaz web visual accesible a través del dominio público, permitiendo a los administradores visualizar el dashboard de métricas en tiempo real y configurar las topologías de red de manera interactiva.

5.2. Modelo de datos

5.2.1. Modelo Entidad-Relación

A continuación se presenta el Modelo Entidad-Relación (MER) del sistema. Este diagrama ilustra el diseño conceptual de la base de datos, identificando las entidades principales, sus atributos clave y las relaciones lógicas establecidas entre ellas, incluyendo las cardinalidades correspondientes

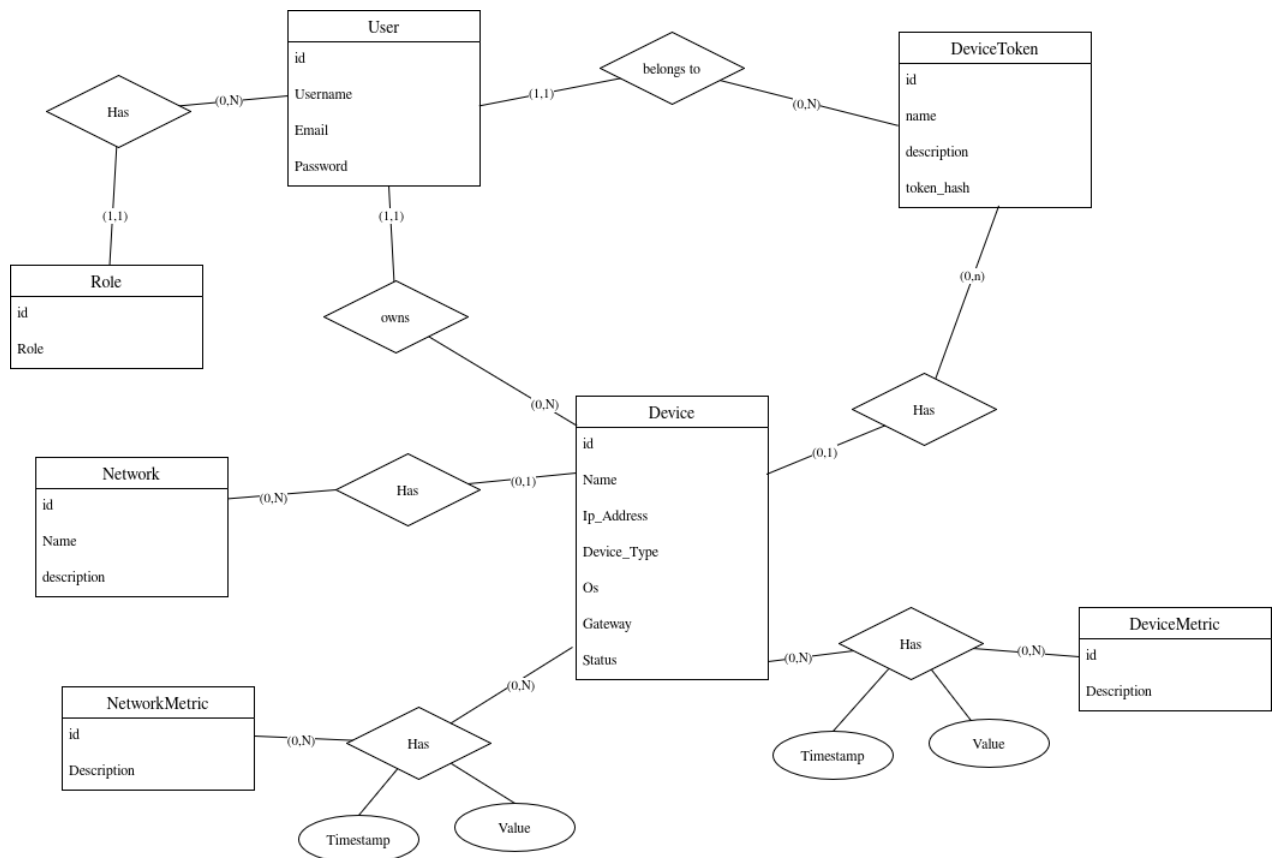


Figura 5.1: Diagrama Entidad-Relación

5.2.2. Modelo Relacional

Derivado del análisis y normalización del diagrama anterior, a continuación se detalla el Modelo Relacional (MR) resultante. En este esquema se definen las estructuras de tablas finales, especificando las claves primarias, la integridad referencial mediante claves foráneas y las tablas transaccionales encargadas de almacenar el histórico de métricas.

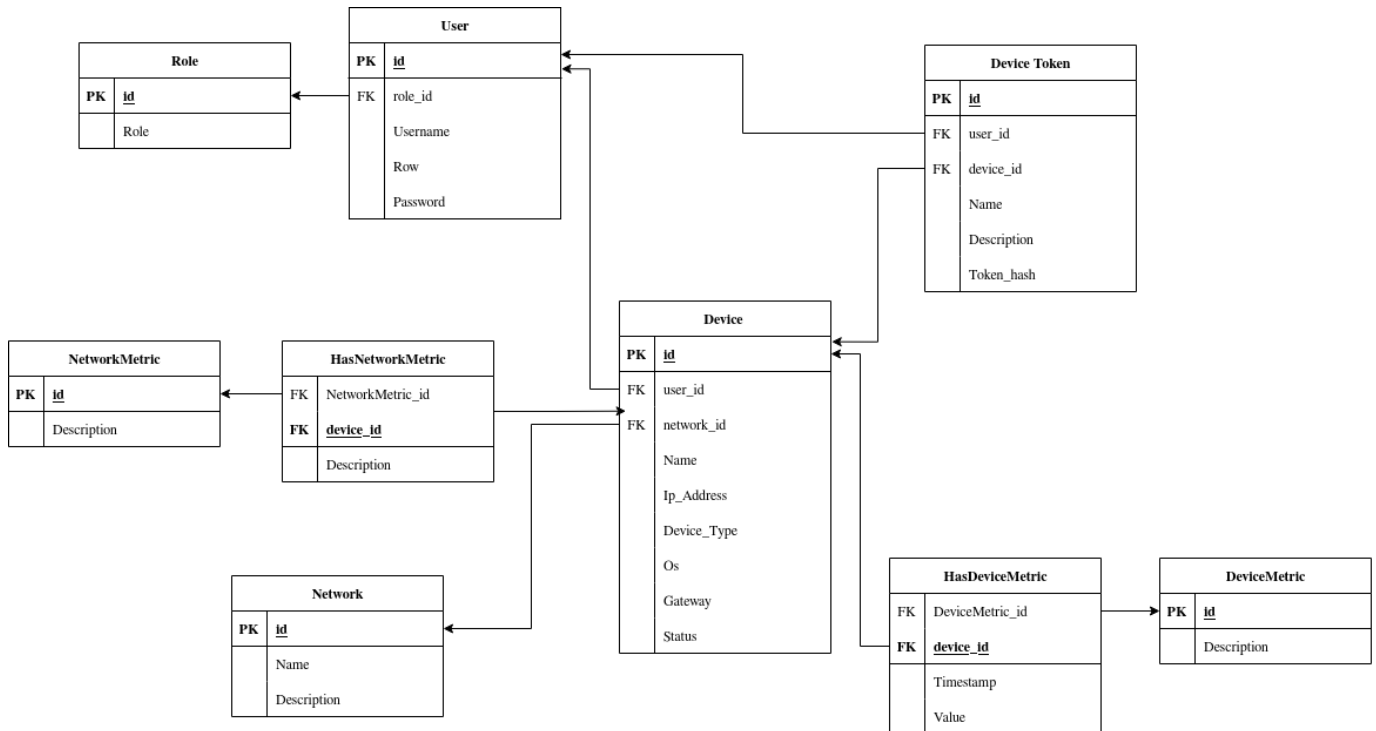


Figura 5.2: Diagrama Entidad-Relación

5.3. Diseño de interfaz y navegación (Mockups)

5.3.1. Guías de estilos

La guía de estilo marcará las pautas a seguir para el diseño de la web. Por tanto, servirá de consulta para visualizar los objetivos de la aplicación en cuanto a su estilo. El principal objetivo es dotar al estilo de la aplicación de la máxima sencillez posible, a fin de que al usuario le resulte intuitivo el uso de la aplicación. A continuación se señalarán aspectos relativos al estilo como son el logotipo, los colores principales y secundarios, la tipografía y la composición de las interfaces.

5.3.2. Logotipo

Podemos distinguir dos identidades visuales claramente diferenciadas que, aunque complementarias, cumplen funciones distintas.

"Nocturne Security": El logotipo en la Figura 5.3 de la marca adopta la silueta de un lobo, una figura elegida por su simbolismo de vigilancia ininterrumpida. Esta imagen hace alusión directa a la facultad de este animal para permanecer alerta y perceptivo, incluso en la oscuridad. Representa una promesa de vigilia permanente (24/7), encarnando el rol de un guardián dedicado a proteger la integridad y seguridad de los dispositivos.



Figura 5.3: Logo Nocturne Security

"Nocturne Scope": Por otro lado, el logotipo del software presente en la Figura 5.4 se materializa en la forma de una llave. Este diseño juega conceptualmente con el título del proyecto ("Scope." alcance), presentando la herramienta no solo como un mecanismo de cierre, sino como un instrumento de acceso y solución. La llave simboliza el punto de entrada estratégico hacia el reforzamiento de las organizaciones, otorgando el control necesario para blindar sus sistemas y desbloquear un nuevo estándar en seguridad informática.



Figura 5.4: Logo Nocturne Scope

5.3.3. Guía de colores

La paleta de colores de la aplicación componen una gama cromática suave, inspirada en la paleta Rosé Pine y orientada a equilibrar tonos oscuros de descanso visual con acentos delicados. En primer lugar, los colores principales que se utilizarán mayoritariamente en el diseño de las interfaces están reunidos en la paleta de la figura 5.5. Por una parte, “Pine” (#31748f) y “Foam” (#9ccfd8) serán los colores de los elementos con los que el usuario podrá interactuar, aportando un contraste fresco y sereno. Y, por otra parte, “Text” (#e0def4) y “Base” (#191724) serán los correspondientes al texto y al fondo de la aplicación, garantizando legibilidad y un entorno agradable al ojo en sesiones prolongadas.

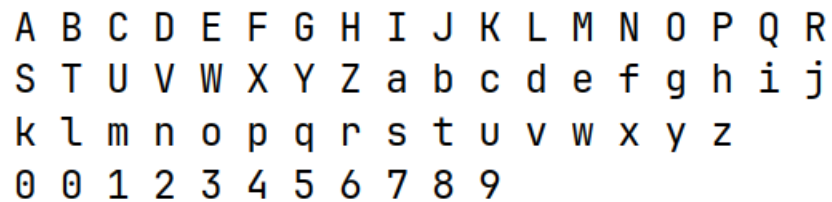
Además de estos colores, se utilizarán otros tonos relacionados con la acción que realizan los botones en los que se apliquen. Se trata de “Love” (#eb6f92) para acciones críticas como “Eliminar” y “Cancelar”, “Gold” (#f6c177) para acciones de modificación como “Editar”, y “Iris” (#c4a7e7) para acciones de confirmación como “Enviar” y “Aceptar”. Estos matices refuerzan la semántica de cada acción mediante un código de color coherente y distintivo.



Figura 5.5: Paleta Rosé Pine

5.3.4. Tipografía

La tipografía utilizada es un tipo de letra monoespaciada diseñada para maximizar la legibilidad y claridad en entornos digitales. Se ha optado por emplear JetBrains Mono, una fuente gratuita desarrollada por JetBrains bajo la licencia SIL Open Font License 1.1, que permite su uso tanto personal como comercial. Esta tipografía ha sido especialmente diseñada para desarrolladores y entornos de programación, pero resulta igualmente adecuada para presentaciones técnicas y documentos donde se prioriza la precisión visual. En la figura se incluye una representación de dicha fuente con los caracteres más comunes.



A B C D E F G H I J K L M N O P Q R
S T U V W X Y Z a b c d e f g h i j
k l m n o p q r s t u v w x y z
0 0 1 2 3 4 5 6 7 8 9

Figura 5.6: Tipografía

5.3.5. Composición de las interfaces

La composición de las interfaces expuesta a continuación corresponde a la versión web de la aplicación, la cual será la única plataforma donde se podrá acceder y administrar el software.

Distinguiremos entre 3 tipos de interfaces según su composición: landing page, topología de red y estadísticas del sistema.

La landing page presenta la plataforma, su propósito y su identidad visual. Incluye un mensaje de bienvenida, una descripción breve del sistema y accesos directos importantes como el repositorio GitHub y la descarga del agente. Su función es introducir al usuario al ecosistema de NocturneScope de forma simple y orientarlo hacia las acciones iniciales.

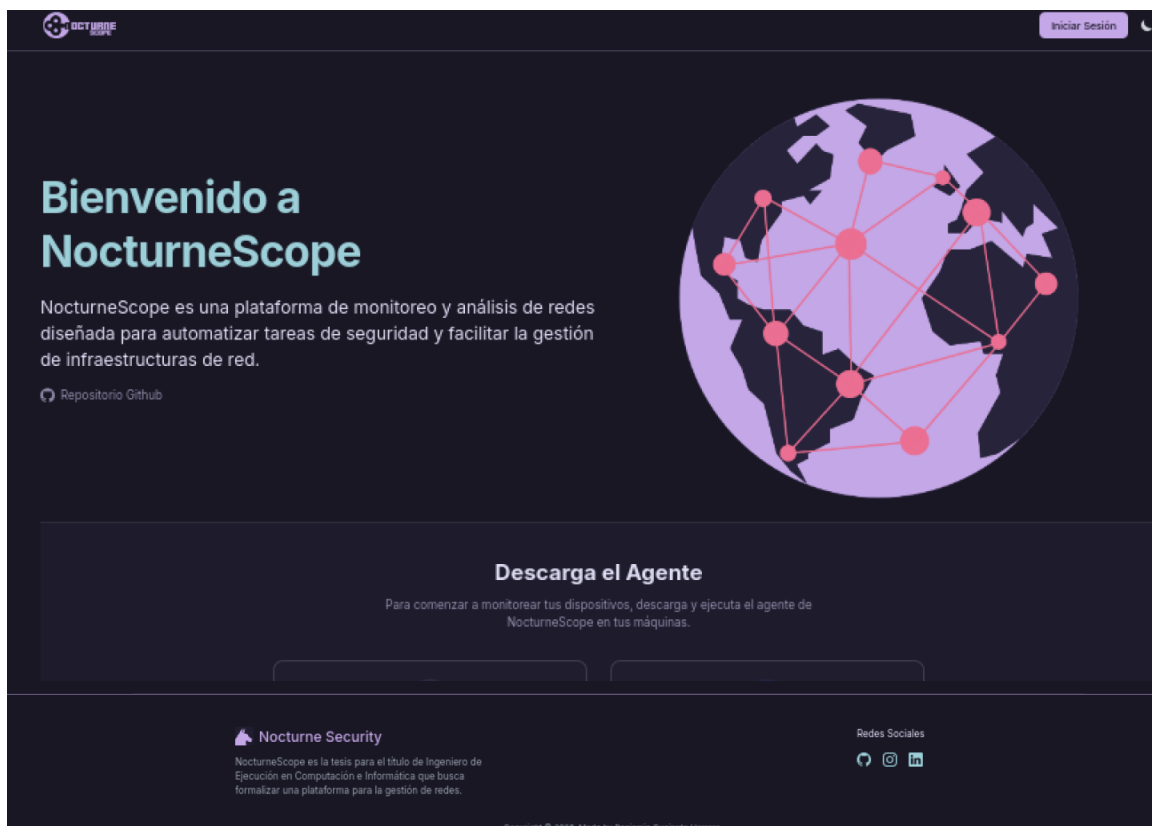


Figura 5.7: Landing Page

El dashboard permite visualizar métricas en tiempo real de los dispositivos registrados. Contiene filtros para seleccionar la métrica, intervalo y rango de tiempo, además de paneles informativos con CPU, RAM, red y temperatura. Su gráfico principal muestra la evolución temporal de los datos, facilitando el análisis rápido del estado del sistema.

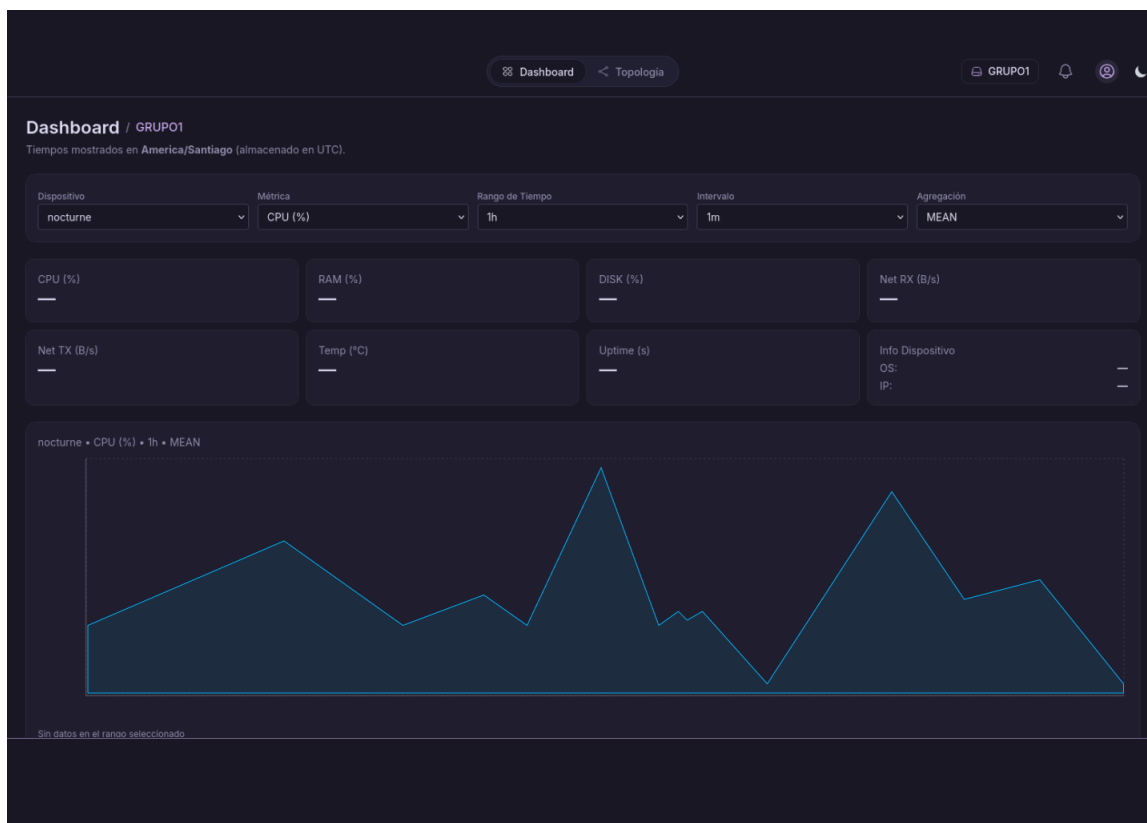


Figura 5.8: Dashboard

La vista de topología funciona como un canvas visual donde se representan dispositivos, herramientas y sus conexiones. Permite crear flujos de monitoreo y automatización arrastrando y enlazando nodos como triggers, notificaciones o acciones. Es la interfaz destinada a construir procesos personalizados dentro de la plataforma.

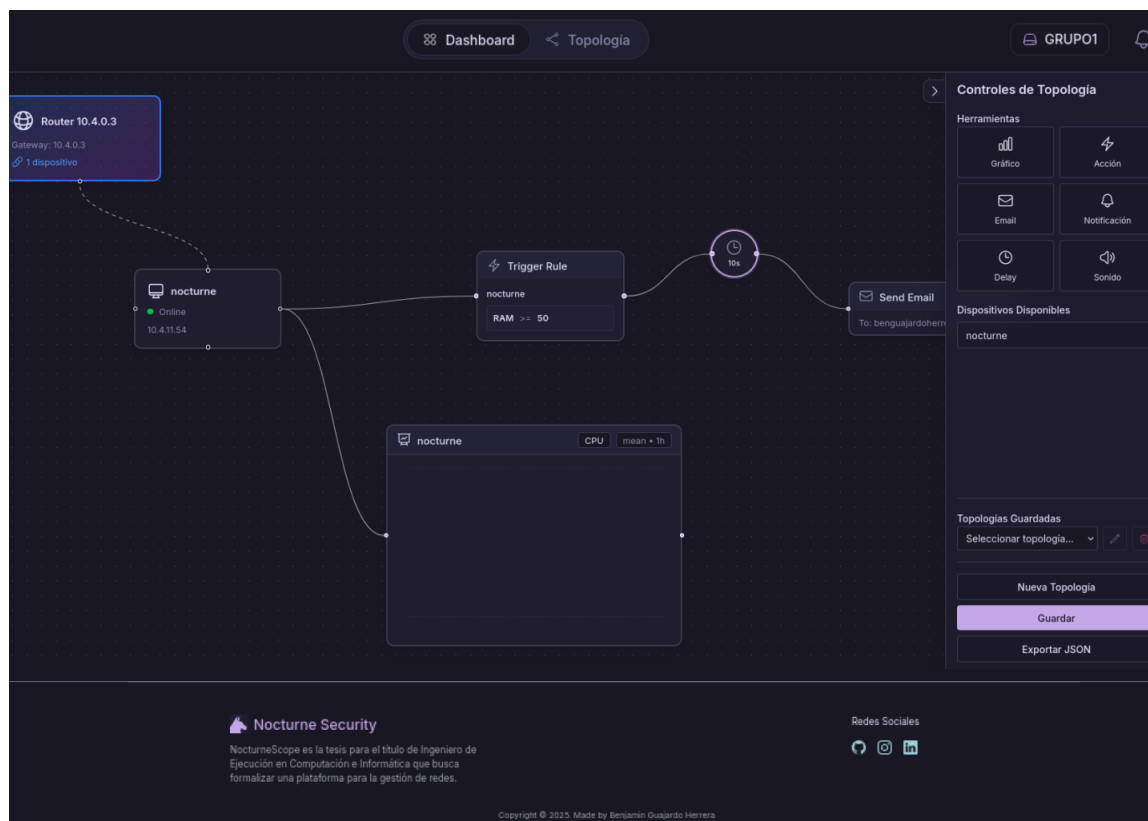


Figura 5.9: Topología

Capítulo 6

Desarrollo del Trabajo

El presente capítulo tiene por objetivo describir el proceso de construcción técnica e implementación de la plataforma "Nocturne Scope", ahondando en la estructura interna del proyecto, tanto en código como infraestructura en red.

6.1. Diseño de arquitectura

Para el diseño de la arquitectura del sistema, tal y como se observa en la Figura 6.1, se ha determinado una estrategia de implementación basada en servidores propios, complementada con tecnologías de tunelización en la nube para garantizar la accesibilidad y seguridad perimetral [12]. Tal como se representa en la ilustración correspondiente, el núcleo de la infraestructura reside en un servidor central denominado "Servidor Nocturne", la cual opera sobre el sistema operativo Debian 13. Este servidor aloja la totalidad de los componentes lógicos mediante una arquitectura de contenedores docker, lo que permite aislar las dependencias y gestionar eficientemente los recursos [13]. Dentro de este entorno virtualizado conviven el Backend, desarrollado en Golang [14], y el Frontend, construido en Javascript, junto a una capa de persistencia de datos híbrida que integra PostgreSQL para la gestión de información relacional y usuarios, e InfluxDB para el almacenamiento optimizado de series temporales y métricas de consumo [15].

Respecto a la conectividad y exposición de servicios, la arquitectura prescinde de una dirección IP pública estática y de la apertura tradicional de puertos en el firewall físico, optando por la implementación de un Cloudflare Tunnel. Este componente establece un enlace seguro cifrado desde el servidor interno hacia la nube, permitiendo asociar dominios públicos directamente a los contenedores internos, específicamente, el dominio `scope.nocturnesec.cl` se enruta hacia la interfaz web (Frontend), mientras que `api.nocturnesec.cl` dirige el tráfico hacia los servicios de Backend. Finalmente, los componentes distribuidos del sistema corresponden a los dispositivos de usuario, los cuales ejecutan un agente de software local. Estos agentes recolectan la telemetría de consumo en las instalaciones del cliente y transmiten los datos vía internet hacia la API del servidor central a través del túnel establecido.

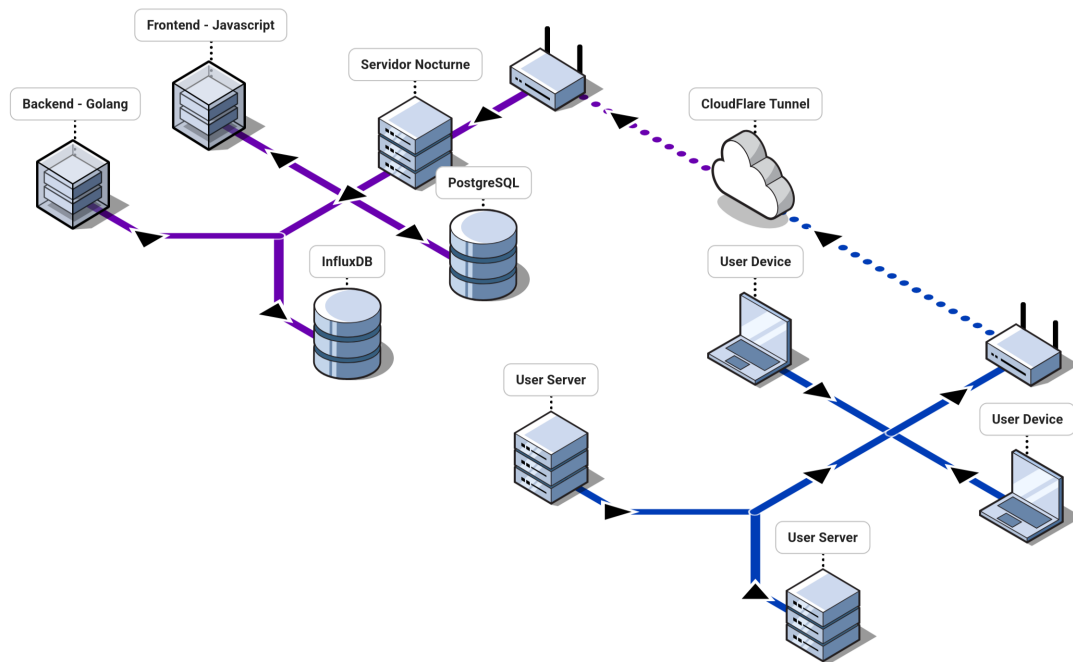


Figura 6.1: Arquitectura en Red del Proyecto

6.2. Estructura del código

El proyecto se organiza en dos grandes componentes principales: **backend** y **frontend**.

Para el Backend, se utiliza el lenguaje de programación Go y se sigue una arquitectura basada en Clean Architecture [11] (Arquitectura Limpia).

Clean Architecture

Clean Architecture propone estructurar un sistema en capas que se organizan desde lo más general a lo más específico. La idea central es proteger la lógica de negocio, evitando que dependa de bases de datos, frameworks, interfaces o cualquier detalle técnico.

La regla clave es simple, el código de las capas externas puede depender de las internas, pero nunca al revés. De este modo, la parte más importante del software (las reglas de negocio) permanece estable y adaptable a futuro.

Las capas, representadas en la Figura 6.2, funcionan de la siguiente manera:

- (a) **Entidades (Dominio)**: Es el núcleo del sistema. Contiene las reglas de negocio empresariales y las estructuras de datos fundamentales. No tiene dependencias externas y es la capa menos propensa a cambios.
- (b) **Casos de Uso (Aplicación)**: Contiene las reglas de negocio específicas de la aplicación. Orquesta el flujo de datos hacia y desde las entidades para cumplir con los objetivos del usuario.
- (c) **Adaptadores de Interfaz**: Su función es convertir los datos entre el formato más conveniente para los casos de uso y el formato requerido por agentes externos (como la web o la base de datos). Aquí residen los controladores y presentadores.
- (d) **Frameworks y Drivers (Infraestructura)**: Es la capa más externa. Contiene los detalles técnicos como la base de datos, el framework web o dispositivos externos. El código aquí solo se comunica hacia adentro a través de los adaptadores.

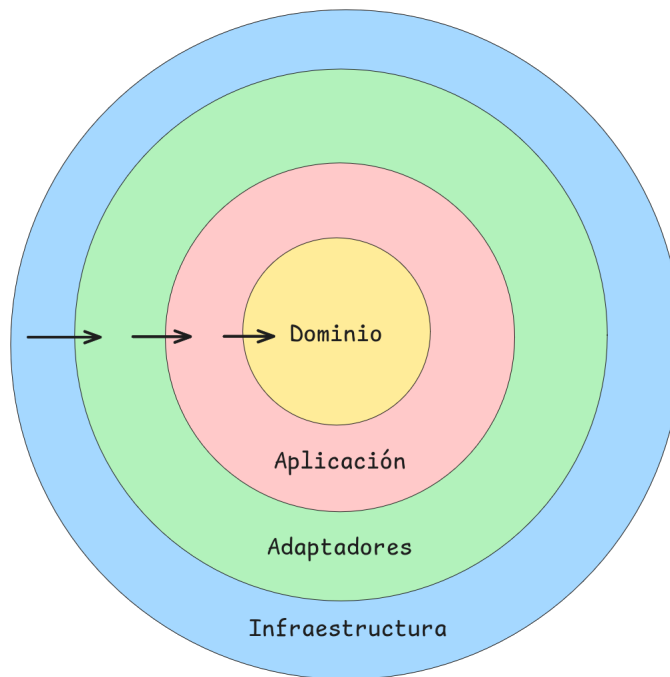


Figura 6.2: Diagrama Clean Architecture

Para el Frontend, se utiliza el framework Next.js (basado en React) con TypeScript. Se emplea el App Router de Next.js para la gestión de rutas y una arquitectura basada en componentes reutilizables, organizados por funcionalidad y dominio.

A continuación se presenta el árbol de directorios general del proyecto:

Estructura de directorios

- NocturneScope/
 - agent/
 - cmd/
 - config/
 - internal/
 - ◊ domain/
 - ◊ infrastructure/
 - ◊ interface/
 - ◊ usecase/
 - backend/
 - cmd/
 - config/
 - internal/
 - ◊ domain/
 - ◊ infrastructure/
 - ◊ interface/
 - ◊ usecase/
 - migrations/
 - frontend/
 - public/
 - src/
 - ◊ app/
 - ◊ components/
 - ◊ context/
 - ◊ lib/

6.2.1. Backend

La estructura del backend está diseñada para desacoplar las dependencias y centrarse en el dominio, respetando las capas de Clean Architecture.

Directorio	Detalle
<code>cmd</code>	Contiene el punto de entrada principal de la aplicación (<code>main.go</code>). Es donde se inicializan las dependencias y se arranca el servidor.
<code>config</code>	Almacena la configuración de la aplicación, incluyendo variables de entorno y parámetros de conexión a servicios externos.
<code>internal/domain</code>	(Capa de Dominio) Define las entidades centrales del negocio y las interfaces (contratos) de los repositorios. Es el núcleo de la aplicación y no tiene dependencias externas.
<code>internal/usecase</code>	(Capa de Aplicación) Contiene la lógica de negocio específica de la aplicación. Orquesta el flujo de datos entre las entidades y los adaptadores, implementando los casos de uso del sistema.
<code>internal/interface</code>	(Capa de Adaptadores) Contiene los controladores HTTP (<code>handlers</code>), middlewares y la definición de rutas. Convierte los datos de la web al formato más conveniente para los casos de uso y viceversa.
<code>internal/- infrastructure</code>	(Capa de Frameworks & Drivers) Implementación técnica de las interfaces definidas en el dominio. Incluye la conexión concreta a bases de datos (PostgreSQL, InfluxDB), clientes de APIs externas y otros servicios de bajo nivel.

Tabla 6.1: Estructura del Backend

Funcionalidad-Endpoints a utilizar:

- (a) 1. <http://api.nocturnesec.cl/api/auth/login>
 - a) a. Tipo de petición:
 - i. “POST”
 - b) b. Parámetros a ingresar, el tipo de datos y restricciones
 - 1) i. Email
 - String
 - Requerido
 - Formato de correo válido (ej. usuario@dominio.com)
 - 2) ii. Password
 - String
 - Requerido
 - Mínimo 6 caracteres
 - Debe contener letras y números
- (b) 2. <http://api.nocturnesec.cl/api/auth/register>
 - a) a. Tipo de petición:
 - i. “POST”
 - b) b. Parámetros a ingresar, el tipo de datos y restricciones
 - 1) i. Username
 - String
 - Requerido
 - Mínimo 3 caracteres, máximo 20
 - Solo caracteres alfanuméricos
 - 2) ii. Email
 - String
 - Requerido
 - Formato de correo válido
 - 3) iii. Role
 - String
 - Requerido
 - Valores permitidos: “admin”, “user”
 - 4) iv. Password
 - String
 - Requerido
 - Mínimo 6 caracteres
 - Debe contener letras y números
- (c) 3. <http://api.nocturnesec.cl/api/topologies>
 - a) a. Tipo de petición:
 - i. “POST”
 - b) b. Parámetros a ingresar, el tipo de datos y restricciones
 - 1) i. Name
 - String

- Requerido
- Mínimo 3 caracteres, máximo 50
- 2) ii. Data
 - JSON (Interface)
 - Requerido
 - Estructura JSON válida representando nodos y enlaces
- (d) 4. <http://api.nocturnesec.cl/api/device-groups>
 - a) a. Tipo de petición:
 - i. “POST”
 - b) b. Parámetros a ingresar, el tipo de datos y restricciones
 - 1) i. Name
 - String
 - Requerido
 - Mínimo 3 caracteres, máximo 50
 - 2) ii. Description
 - String
 - Opcional
 - Máximo 200 caracteres
- (e) 5. <http://api.nocturnesec.cl/api/network-traffic>
 - a) a. Tipo de petición:
 - i. “POST”
 - b) b. Parámetros a ingresar, el tipo de datos y restricciones
 - 1) i. device_id
 - String
 - Requerido
 - UUID válido del dispositivo
 - 2) ii. Protocol
 - String
 - Requerido
 - Valores permitidos: “TCP”, “UDP”
 - 3) iii. Source_Ip
 - String
 - Requerido
 - Formato IPv4 o IPv6 válido
 - 4) iv. Destination_Port
 - Integer
 - Requerido
 - Rango: 0 - 65535
 - 5) v. Connection_State
 - String
 - Requerido
 - Ej: “ESTABLISHED”, “LISTEN”, “TIME_WAIT”

- 6) vi. Threat_level
 - String
 - Opcional (Default: “LOW”)
 - Valores: “LOW”, “MEDIUM”, “HIGH”, “CRITICAL”
- 7) vii. Duration
 - Integer
 - Opcional
 - Valor en segundos ≥ 0
- 8) viii. Timestamp
 - String
 - Requerido
 - Formato ISO 8601 (YYYY-MM-DDTHH:MM:SSZ)

6.2.2. Frontend

El frontend sigue la estructura estándar de Next.js con App Router, optimizada para escalabilidad y modularidad.

Directorio	Detalle
<code>src/app</code>	Contiene las páginas de la aplicación y define la estructura de rutas (routing). Cada carpeta representa una ruta URL (ej. <code>/dashboard</code> , <code>/auth/login</code>).
<code>src/components</code>	Biblioteca de componentes de interfaz de usuario reutilizables (botones, tablas, gráficos, formularios). Se organizan por funcionalidad (ej. <code>admin</code> , <code>dashboard</code> , <code>topology</code>).
<code>src/context</code>	Implementación de contextos de React para el manejo del estado global de la aplicación (ej. autenticación, preferencias de usuario).
<code>src/lib</code>	Funciones de utilidad, helpers, constantes y configuraciones compartidas (ej. cliente HTTP, funciones de formateo).
<code>public</code>	Archivos estáticos accesibles públicamente, como imágenes, íconos y fuentes.

Tabla 6.2: Estructura del Frontend

Capítulo 7

Implantación y Puesta en Marcha

El presente capítulo describe el proceso planificado para la adopción del sistema Nocturne Scope en un entorno organizacional, abarcando la estrategia de implantación, las actividades de capacitación y el proceso operativo previo a la puesta en marcha.

7.1. Plan de Capacitación/Entrenamiento

La adopción efectiva del sistema requiere que los usuarios comprendan su arquitectura, flujo operativo y las funcionalidades asociadas al monitoreo, la visualización y la automatización. Considerando que los perfiles objetivo poseen nociones en administración de sistemas y herramientas de monitoreo, se propone un programa de capacitación estructurado en módulos independientes y progresivos, orientado a fortalecer la autonomía del usuario y facilitar la integración del software en la infraestructura tecnológica. Cada módulo aborda una etapa crítica del uso de la plataforma, combinando teoría y ejemplos prácticos. Estos contenidos estarán disponibles como un curso online dentro de la documentación oficial, y para organizaciones que lo requieran, se contempla además la posibilidad de impartir cursos presenciales de capacitación.

Módulo 1: Introducción Conceptual y Arquitectura General

Este módulo presenta los fundamentos esenciales para comprender el funcionamiento de "Nocturne Scope" dentro del entorno organizacional. Se explica la arquitectura general del sistema, describiendo el rol del agente instalado en los dispositivos, las funciones del backend y la estructura de la interfaz web utilizada para visualizar la información. Además, se detalla el flujo de datos desde la captura de métricas hasta su representación gráfica, junto con la forma en que el servidor central coordina el monitoreo, la seguridad de la comunicación y la generación de eventos. El propósito de este módulo es que los usuarios obtengan una visión clara de cómo se articulan los componentes del sistema y cómo estos influyen en la estabilidad y el rendimiento del monitoreo en tiempo real.

Módulo 2: Instalación, Configuración y Operación del Agente

En esta etapa se instruye a los usuarios en el proceso de instalación del agente en dispositivos Linux, abarcando los requisitos técnicos necesarios, la configuración inicial y las validaciones posteriores. Se explica de forma detallada el uso de tokens de API, que permiten la vinculación segura entre los dispositivos y la plataforma, junto con las implicancias de su vigencia, permisos y regeneración. Asimismo, se describe el procedimiento para verificar la conexión con el servidor y confirmar que las métricas están siendo transmitidas correctamente. Este módulo es especialmente relevante, ya que la correcta instalación del agente determina la calidad y continuidad del monitoreo, por lo que se entregan ejemplos prácticos y pasos secuenciales que facilitan su ejecución.

Módulo 3: Uso de la Interfaz Web y Gestión Operativa

Una vez que los dispositivos están vinculados y activos, la capacitación se orienta al uso de la interfaz web, donde se revisan las herramientas de visualización y administración disponibles para los usuarios. Se explica cómo interpretar las métricas operativas de cada dispositivo, tales como el uso de CPU, memoria, temperatura, tráfico de red y estado general. También se enseña a administrar dispositivos registrados, actualizar su información y utilizar la vista nodal para organizar la topología de la infraestructura. La interfaz gráfica se presenta como una herramienta fundamental para comprender la relación entre equipos, identificar dependencias operativas y facilitar la toma de decisiones mediante un diseño visual que reduce la complejidad del monitoreo tradicional.

Módulo 4: Creación de Automatizaciones y Definición de Flujos

Este módulo profundiza en la construcción de flujos de automatización utilizables dentro del sistema, explicando los conceptos relacionados con disparadores, condiciones lógicas y acciones ejecutables. Se instruye a los usuarios en la configuración de umbrales para métricas críticas, la detección de eventos relevantes y la forma en que se pueden definir respuestas automáticas, como notificaciones o acciones remotas. Se hace especial énfasis en el diseño responsable de automatizaciones, considerando que algunas acciones impactan directamente en los dispositivos monitoreados y requieren autenticación adicional para evitar errores operativos. Además, se incorporan ejemplos de flujos típicos, permitiendo que los usuarios comprendan los beneficios y riesgos de automatizar procesos dentro de la infraestructura.

Módulo 5: Generación de Reportes e Interpretación del Historial de Eventos

El último módulo aborda las herramientas de análisis histórico disponibles dentro de la plataforma. Se explica cómo generar reportes basados en rangos temporales, dispositivos seleccionados y tipos de métricas específicas, entregando criterios claros para su uso en auditorías, diagnósticos y planificación operativa. Asimismo, se profundiza en el uso del historial de eventos para revisar patrones de comportamiento, detectar fallas recurrentes y evaluar la evolución temporal del rendimiento de la infraestructura. Este módulo destaca la importancia del registro persistente

de información para la toma de decisiones estratégicas y como elemento de apoyo en procesos formales de documentación interna.

7.2. Estrategia de Implantación

La implantación de "Nocturne Scope" en un entorno empresarial requiere una transición ordenada y validada, capaz de adaptarse a la infraestructura tecnológica existente sin interrumpir las operaciones críticas. Para ello, se propone una estrategia basada en un despliegue piloto, orientada a evaluar el comportamiento del sistema en condiciones reales, verificar la interoperabilidad entre sus componentes e introducir progresivamente a los usuarios al nuevo modelo de monitoreo y automatización. Esta estrategia constituye un ejemplo de referencia aplicable a empresas que decidan implementar la plataforma durante el mes de enero del próximo año, pudiendo ajustarse según el tamaño de la organización y la complejidad de su red.

Paso 1: Preparación del Entorno y Validación Técnica

La primera etapa se orienta a preparar un entorno operativo estable donde se desplegará "Nocturne Scope". En esta fase se configura el servidor principal utilizando contenedores Docker, se habilita un túnel seguro mediante Cloudflare y se inicializan las bases de datos PostgreSQL e InfluxDB. Posteriormente se ejecuta una verificación técnica completa, asegurando que los servicios puedan comunicarse entre sí y que la plataforma sea capaz de recibir y almacenar métricas de prueba. Esta validación temprana permite detectar posibles incompatibilidades, evaluar la carga inicial y garantizar que la infraestructura se encuentra en condiciones adecuadas para iniciar el piloto.

Paso 2: Selección del Conjunto Piloto

Una vez validado el entorno técnico, se selecciona un conjunto representativo de dispositivos para iniciar la fase piloto. Para asegurar un volumen adecuado de información, se recomienda incluir aproximadamente diez dispositivos por departamento, equilibrando entre servidores y estaciones de trabajo. En este módulo se designa también un Encargado TI que actúa como enlace operativo entre la empresa y el equipo responsable del sistema, canalizando dudas, incidentes y observaciones durante el proceso. Una selección adecuada del conjunto piloto es clave para obtener datos reales y evaluar el comportamiento del sistema de forma precisa.

Paso 3: Ejecución del Piloto en Entorno Real

El piloto constituye la fase central de la implantación. Durante un periodo aproximado de dos semanas, se habilitan las principales funcionalidades de la plataforma, incluyendo la visualización de topología, el monitoreo en tiempo real, la gestión de dispositivos registrados y la ejecución de automatizaciones básicas. Los agentes instalados comienzan a transmitir métricas de forma continua, lo que permite evaluar la estabilidad del servidor, la frecuencia de actualización del panel web y la consistencia de los datos almacenados. La organización obtiene así una primera visión del rendimiento del sistema y puede identificar necesidades de ajuste sin comprometer la totalidad de su infraestructura.

Paso 4: Ajustes, Observaciones y Retroalimentación

Al concluir el piloto, se recopila la información operativa registrada durante el periodo de prueba. Las incidencias reportadas por los usuarios, junto con los registros técnicos del sistema, permiten identificar áreas de mejora y priorizar correcciones. El Encargado TI entrega un reporte consolidado que incluye observaciones sobre la interfaz, el comportamiento de las automatizaciones, la claridad de las métricas y la estabilidad del dashboard. Con esta retroalimentación, el equipo responsable aplica las mejoras necesarias para optimizar la experiencia de uso y asegurar que la plataforma esté en condiciones de operar a escala completa.

Paso 5: Puesta en Marcha Formal

La última etapa corresponde a la activación completa del sistema dentro de la organización. Tras la aplicación de los ajustes derivados del piloto, se procede a habilitar los agentes en todos los dispositivos objetivo, validar la topología general y revisar las reglas de automatización activas para evitar ejecuciones no deseadas. Finalmente, se verifica la estabilidad del dashboard y del almacenamiento histórico, garantizando que la organización cuente con una plataforma confiable para el monitoreo continuo de su infraestructura TI. Con ello, el sistema queda completamente integrado en las operaciones diarias.

Cronograma Referencial - Implementación en Enero y Febrero del Próximo Año

El siguiente cronograma representa una planificación referencial de implantación distribuida en dos meses (enero y febrero del próximo año). Se organiza por semanas, de manera que la empresa pueda ejecutar la preparación técnica, el piloto y la puesta en marcha de forma progresiva y controlada. Las fechas pueden adaptarse según la magnitud del despliegue y la disponibilidad del personal.

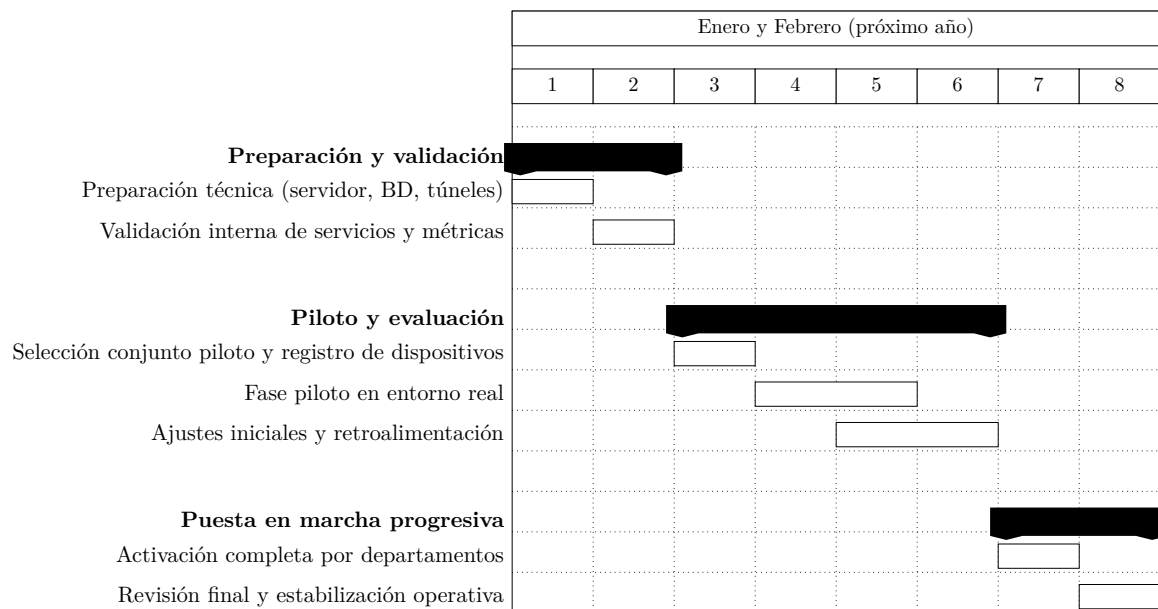


Figura 7.1: Cronograma referencial de implementación del sistema durante dos meses (enero y febrero del próximo año). Las columnas 1 a 8 representan semanas consecutivas.

Capítulo 8

Conclusiones

El desarrollo del proyecto “Nocturne Scope” ha permitido consolidar una solución tecnológica que responde de manera efectiva a la problemática de la complejidad y la falta de automatización en la gestión de infraestructura TI. A través de la implementación de una arquitectura moderna y escalable, se ha logrado materializar una plataforma que no solo compite en funcionalidad con herramientas tradicionales, sino que mejora significativamente la experiencia de usuario y la eficiencia operativa mediante un enfoque de visualización intuitiva.

Respecto al logro de los objetivos planteados, se concluye que el objetivo general del software ha sido alcanzado satisfactoriamente, logrando monitorear la infraestructura en tiempo real mediante una visualización interactiva. En cuanto a los objetivos específicos del software, se logró generar exitosamente la representación de nodos en la topología de red, permitiendo visualizar métricas generales y comprender el estado de la infraestructura. Asimismo, se cumplió con la capacidad de generar alertas sobre sucesos importantes y mantener la trazabilidad de incidentes. La integración del agente para la recolección de métricas y su envío seguro se completó con éxito, garantizando la comunicación fluida con la plataforma central. Finalmente, se materializó la automatización de tareas basada en series de acciones configurables.

En relación con el tiempo y el esfuerzo invertido, la planificación se ajustó a la realidad de la implementación, aunque con márgenes acotados que exigieron un sobre esfuerzo en las últimas semanas de desarrollo. Si bien la estimación de esfuerzo para el despliegue fue precisa, la etapa de desarrollo presentó desafíos técnicos importantes, siendo el principal obstáculo la selección y optimización del lenguaje para el *backend*. La decisión estratégica de migrar hacia Go (Golang) representó un bloqueo inicial debido a la curva de aprendizaje, pero fue determinante para el éxito del proyecto.

Es importante señalar que el desarrollo de este software constituyó una instancia fundamental para validar las competencias del perfil de egreso, destacando especialmente la capacidad de auto-aprendizaje y adaptación a tecnologías emergentes. Gran parte del *stack* tecnológico, incluyendo Go, Fiber, Next.js e InfluxDB, fue adquirido de manera autodidacta durante el proceso, demostrando la motivación por el aprendizaje permanente. Además, se logró construir una solución informática efectiva que simplifica drásticamente la gestión TI en comparación con estándares de la industria, eliminando la necesidad de configuraciones manuales complejas y facilitando la adopción por parte de los usuarios.

Finalmente, desde una perspectiva personal, el proyecto superó las expectativas académicas iniciales, perfilándose como un producto con viabilidad comercial real. La experiencia de ver la transición de la teoría a la práctica, con la automatización ejecutándose exitosamente y el monitoreo operando en tiempo real sobre dispositivos físicos, resultó sumamente gratificante, confirmando que la ingeniería aplicada es capaz de otorgar control y tranquilidad en la operación de infraestructura crítica.

8.1. Trabajo Futuro

Considerando que la base tecnológica del sistema ya se encuentra operativa y estable, las líneas de trabajo futuro se orientan principalmente a la escalabilidad de los módulos funcionales:

- **Ampliación del catálogo de automatización:** Desarrollar nuevos módulos predefinidos que abarquen un rango más amplio de acciones correctivas automáticas, reduciendo aún más la intervención manual.
- **Interoperabilidad con herramientas de terceros:** Implementar conectores que permitan la integración con soluciones externas, con especial énfasis en herramientas de ciberseguridad, facilitando la correlación de eventos y fortaleciendo la postura de seguridad de la organización.
- **Integración de Inteligencia Artificial:** Incorporar análisis predictivo sobre los datos históricos almacenados en InfluxDB para anticipar fallas antes de que ocurran, permitiendo la transición de un modelo de soporte reactivo a uno preventivo.
- **Integración con dispositivos de red:** Extender la capacidad de monitoreo más allá de servidores y estaciones de trabajo para incluir soporte nativo a routers y switches, permitiendo así una visualización completa y centralizada de la topología de red.

Referencias

- [1] United Nations Department of Economic and Social Affairs, “Regional e-government development and the performance of country groupings,” 2024.
- [2] World Economic Forum, “Future of jobs report 2023,” May 2023.
- [3] Opendgear, “The many costs of downtime,” 2022.
- [4] Cisco Systems, “Network automation trends and strategy,” 2021.
- [5] S. Puuska, J. Karjalainen, R. Savola, and P. Kujala, “Nationwide critical infrastructure monitoring – situation awareness oriented design,” *International Journal of Critical Infrastructure Protection*, vol. 23, pp. 76–87, 2018.
- [6] Nagios Enterprises, LLC. (2025) Nagios. Consultado: 01 sep. 2025. [Online]. Available: <https://www.nagios.org/>
- [7] Zabbix LLC. (2025) Zabbix. Consultado: 15 sep. 2025. [Online]. Available: <https://www.zabbix.com/>
- [8] ScienceLogic, Inc. (2025) Sciencelogic sl1. Consultado: 15 sep. 2025. [Online]. Available: <https://sciencelogic.com/product/sl1-platform>
- [9] NetBox Project. (2025) Netbox. Consultado: 04 sep. 2025. [Online]. Available: <https://netbox.dev/>
- [10] R. S. Pressman, *Ingeniería del software: un enfoque práctico*, 7th ed. McGraw-Hill Interamericana, 2010.
- [11] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Pearson Education, 2017.
- [12] Cloudflare Inc., “Reference architecture for internet-native transformation,” Cloudflare, Whitepaper, 2023, rEV:PMM-MAR2023.
- [13] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.

-
- [14] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Addison-Wesley Professional, 2015.
 - [15] InfluxData, *The InfluxDB Storage Engine and the Time-Structured Merge Tree (TSM)*, 2025, accessed: 2025-11-24. [Online]. Available: https://docs.influxdata.com/influxdb/v1/concepts/storage_engine/
 - [16] R. Hertzog and R. Mas, *El manual del administrador de Debian: Debian Buster desde el descubrimiento al dominio*. Freexian SARL, 2020, disponible en línea. [Online]. Available: <https://debian-handbook.info/browse/es-ES/stable/>
 - [17] Ubuntu Documentation Team, *Ubuntu Server Guide*, Canonical Ltd., 2024, accedido: 2024-01-15. [Online]. Available: <https://ubuntu.com/server/docs>

Apéndice A

Definiciones y abreviaciones del Negocio

Agente: Software ligero desarrollado para ser instalado en los dispositivos del cliente (servidores o estaciones de trabajo Linux). Su función principal es recolectar métricas de telemetría local (CPU, RAM, temperatura) y enviarlas de forma segura al servidor central de Nocturne Scope mediante el uso de tokens de autenticación.

Backend: Componente del servidor desarrollado en el lenguaje de programación Go (utilizando el framework Fiber). Es el encargado de procesar la lógica de negocio, gestionar la base de datos, validar los tokens de los agentes y exponer la API para el frontend.

Frontend: Componente de la interfaz de usuario desarrollado con Next.js y React. Es la parte visual de la plataforma con la que interactúan el Encargado TI y el Equipo TI para ver métricas y gestionar la red.

Canvas (Lienzo): Área de trabajo interactiva dentro de la interfaz web donde el usuario visualiza, arrastra y conecta los nodos. Es el espacio principal para el diseño de la topología de red y la configuración visual de las reglas de automatización.

Clean Architecture (Arquitectura Limpia): Patrón de diseño de software utilizado en el backend del proyecto que estructura el código en capas concéntricas (Entidades, Casos de Uso, Adaptadores, Infraestructura). Su objetivo es aislar la lógica de negocio de las herramientas externas como bases de datos o frameworks web.

Cloudflare Tunnel: Tecnología de tunelización utilizada en la infraestructura del proyecto para exponer los servicios locales (Frontend y API) a internet de forma segura. Permite el acceso público sin necesidad de abrir puertos en el firewall físico ni exponer la dirección IP real del servidor.

Dashboard: Panel de control principal de la aplicación web que visualiza en tiempo real las métricas consolidadas de los dispositivos monitoreados. Utiliza gráficos dinámicos para mostrar la evolución temporal del rendimiento del sistema.

Endpoint: Punto final de comunicación en la API del sistema (ej. `/api/auth/login`). Es la dirección específica a la que el Frontend o los Agentes envían solicitudes para interactuar con el Backend.

Latencia: Tiempo que tarda un paquete de datos en viajar desde un punto a otro. En el contexto de los Requerimientos No Funcionales, se busca minimizar la latencia de carga del dashboard para asegurar una experiencia de usuario fluida.

Métricas: Datos cuantitativos recolectados por el agente (ej. porcentaje de uso de CPU, temperatura en grados Celsius, tráfico de red en Kbps) que permiten evaluar la salud y el rendimiento de un dispositivo.

Nodo: Representación gráfica y visual de un elemento dentro de la vista de topología. Puede representar un dispositivo físico (servidor, PC) o una herramienta lógica (regla de alerta, notificación) y se conecta con otros nodos mediante enlaces.

Series Temporales: Modelo de datos donde la información se almacena asociada a una marca de tiempo (timestamp). Es el formato fundamental para guardar el historial de rendimiento de los dispositivos en InfluxDB.

Token de Vinculación: Cadena alfanumérica única y segura generada por el Encargado TI. Sirve como llave de autorización para que un Agente recién instalado pueda registrarse y comenzar a enviar datos a la plataforma.

Topología: Representación visual y esquemática de la infraestructura de red gestionada. En Nocturne Scope, permite ver la disposición de los dispositivos y sus conexiones lógicas a través de un diagrama de nodos interactivo.

Trigger (Disparador): Regla lógica configurada por el usuario en el módulo de automatización (ej. “Si Temperatura > 80°C”). Al cumplirse la condición, el sistema ejecuta automáticamente una acción predefinida, como enviar una alerta.

Apéndice B

Recopilación de Información

El análisis documental tuvo como objetivo establecer el estado del arte en la gestión de infraestructura TI, identificar los costos asociados a las fallas y evaluar las limitaciones de las herramientas existentes. Se revisaron informes de industria, documentación técnica de software y estándares de operación.

Tabla B.1: Resumen de fuentes documentales revisadas.

Tipo de Fuente	Documento / Herramienta	Información Extraída / Hallazgo
Informes de Industria	<i>The Many Costs of Downtime</i> (Opendgear) [3]	Impacto económico de la indisponibilidad de red y tiempos promedio de respuesta (11 horas).
	<i>Network Automation Trends</i> (Cisco) [4]	Prevalencia de procesos manuales (95 %) en la configuración de redes.
Documentación Técnica de Software	Documentación oficial de Nagios [6] y Zabbix [7]	Identificación de la complejidad en la configuración basada en archivos de texto y sobrecarga de alertas.
	Documentación de NetBox [9]	Análisis de modelos de inventario de red y gestión de direcciones IP.
Protocolos y Manuales	Manuales de administración de servidores Linux Debian [16] y Ubuntu [17] .	Identificación de comandos manuales recurrentes (ej. <code>htop</code> , <code>systemctl</code>) que requieren automatización.

B.2. Observación Directa (Trabajo de Campo)

Se realizó observación y análisis de dos entornos operativos reales con el fin de comprender el flujo de trabajo diario de los administradores de sistemas que no son evidentes en la teoría.

Entornos de Observación

La recolección de datos se llevó a cabo en los siguientes escenarios:

- (a) **Servidor Local del Grupo de Cuántica (Universidad):** Entorno académico/investigación con recursos limitados y alta dependencia de servicios específicos.
- (b) **ASMAR (Astilleros y Maestranzas de la Armada):** Entorno corporativo/industrial durante el desarrollo de la práctica profesional, caracterizado por una infraestructura crítica y protocolos estrictos.

Registro de Hallazgos

La siguiente tabla resume los patrones de comportamiento y problemas detectados en ambos entornos, los cuales fundamentaron los requisitos funcionales del sistema.

Dimensión Observada	Descripción del Problema Detectado
Fragmentación de Herramientas	Se observó una alta dispersión en las herramientas de monitoreo. Los encargados debían cambiar constantemente entre consolas de terminal, interfaces web de proveedores y hojas de cálculo para tener una visión completa del estado de la red.
Digestión de Datos	Datos crudos y poco procesables: La información se presentaba mayoritariamente como logs de texto o métricas aisladas sin contexto visual. Esto obligaba al operador a realizar un esfuerzo cognitivo alto para interpretar si un valor representaba una falla o un comportamiento normal.
Procesos Manuales	Las revisiones de estado se realizaban de forma reactiva y manual. Ante una sospecha de falla, el procedimiento estándar implicaba conectarse vía SSH y ejecutar comandos de diagnóstico uno por uno, aumentando el tiempo de resolución (MTTR).
Visualización de Topología	Ausencia de un mapa visual actualizado de la red. La relación entre dispositivos (quién conecta con quién) residía principalmente en la memoria de los operadores o en diagramas estáticos desactualizados, dificultando el diagnóstico de fallas en cadena.

Tabla B.2: Ficha de hallazgos mediante observación directa.

B.3. Conclusión de la Recopilación

La triangulación de la información obtenida mediante la revisión bibliográfica y la observación en terreno permitió concluir que, transversalmente (tanto en entornos académicos como corporativos), existe una carencia de herramientas que unifiquen la visualización y la automatización.

Estos hallazgos validaron la necesidad de desarrollar *Nocturne Scope* con un enfoque prioritario en:

- Centralizar la información en un único Dashboard (para resolver la dispersión).
- Utilizar una interfaz gráfica de nodos (para resolver la falta de topología visual).
- Implementar automatización de alertas (para reducir la dependencia de revisiones manuales).

Apéndice C

Diccionario de datos

A continuación, se describen las estructuras de datos para la base de datos relacional (PostgreSQL) y el esquema de series temporales (InfluxDB).

C.1. Base de Datos Relacional (PostgreSQL)

Esta sección detalla las entidades encargadas de gestionar la lógica de negocio, usuarios, control de acceso y configuración visual de las topologías.

Atributo	Tipo	Restricción	Descripción
id	UUID	PK, Not Null	Identificador único del usuario.
username	VARCHAR(50)	Unique	Nombre de usuario para identificación en plataforma.
email	VARCHAR(100)	Unique	Correo electrónico para autenticación y notificaciones.
password_hash	VARCHAR(255)	Not Null	Contraseña cifrada (hash).
role	VARCHAR(20)	Check	Rol del usuario: <code>ádmín</code> o <code>úser</code> .
created_at	TIMESTAMP	Default Now	Fecha de creación de la cuenta.

Tabla C.1: Entidad: Usuarios (users)

Atributo	Tipo	Restricción	Descripción
id	UUID	PK, Not Null	Identificador único del dispositivo (generado por el agente).
name	VARCHAR(100)	Not Null	Nombre legible asignado al dispositivo (Hostname).
ip_address	INET	Not Null	Dirección IP detectada del dispositivo.
os_info	VARCHAR(100)	Nullable	Información del Sistema Operativo (ej. Ubuntu 22.04).
status	VARCHAR(20)	Default <code>offline</code>	Estado actual: <code>online</code> , <code>offline</code> , <code>warning</code> .
group_id	UUID	FK	Referencia al grupo al que pertenece el dispositivo.
user_id	UUID	FK	Referencia al usuario propietario del dispositivo.

Tabla C.2: Entidad: Dispositivos (devices)

Atributo	Tipo	Restricción	Descripción
id	UUID	PK, Not Null	Identificador único de la topología.
name	VARCHAR(100)	Not Null	Nombre descriptivo de la red diseñada.
layout_data	JSONB	Not Null	Estructura JSON que almacena la posición (x,y) de los nodos y las conexiones (edges) para la librería gráfica (React Flow).
created_at	TIMESTAMP	Default Now	Fecha de creación del diseño.
user_id	UUID	FK	Usuario creador de la topología.

Tabla C.3: Entidad: Topologías (topologies)

Atributo	Tipo	Restricción	Descripción
id	UUID	PK, Not Null	Identificador del token.
name	VARCHAR(50)	Not Null	Alias para identificar el token (ej. "Servidores Web").
token_key	VARCHAR(255)	Unique	Clave generada para la autenticación de agentes.
expires_at	TIMESTAMP	Not Null	Fecha de expiración de la credencial.
user_id	UUID	FK	Usuario que generó el token.

Tabla C.4: Entidad: Tokens de Acceso (api_tokens)

C.2. Base de Datos de Series Temporales (InfluxDB)

Esta sección describe las mediciones (measurements) utilizadas para almacenar el historial de rendimiento y tráfico de red. A diferencia del modelo relacional, aquí se utilizan *Tags* (índices) y *Fields* (valores).

Campo	Clasificación	Tipo Dato	Descripción
timestamp	Time	Int64	Momento exacto de la captura del dato (precisión nanosegundos).
device_id	Tag	String	ID del dispositivo (índice para búsquedas rápidas).
host	Tag	String	Nombre del host.
cpu_usage	Field	Float	Porcentaje de uso de CPU (0-100).
ram_usage	Field	Float	Porcentaje de uso de Memoria RAM.
disk_usage	Field	Float	Porcentaje de ocupación de disco.
temperature	Field	Float	Temperatura del CPU en grados Celsius.

Tabla C.5: Medición: Telemetría del Sistema (system_metrics)

Campo	Clasificación	Tipo Dato	Descripción
timestamp	Time	Int64	Momento del evento de red.
device_id	Tag	String	Dispositivo que reporta el tráfico.
protocol	Tag	String	Protocolo de transporte detectado (TCP/UDP).
connection_state	Tag	String	Estado de la conexión (ej. ESTABLISHED, LISTEN).
source_ip	Field	String	Dirección IP de origen del paquete.
dest_port	Field	Integer	Puerto de destino.
packet_count	Field	Integer	Cantidad de paquetes transmitidos en el intervalo.
threat_level	Field	String	Nivel de amenaza calculado (LOW, MEDIUM, HIGH).

Tabla C.6: Medición: Tráfico de Red (network_traffic)

Apéndice D

Aspectos de gestión de proyectos

D.1. Carta Gantt con línea base y desviaciones

La figura presentada a continuación refleja la ejecución real del proyecto, contrastando con la línea base planificada. La principal desviación se observa en el **PT 2**, donde la curva de aprendizaje asociada a la implementación de la tecnología *Go (Golang)* requirió la inserción de una etapa no prevista de autoaprendizaje intensivo durante el mes de octubre.

Esta situación generó un desplazamiento en cadena de las actividades de desarrollo del Backend, reduciendo significativamente la holgura disponible para las etapas posteriores. Como consecuencia, Backend y Frontend debieron ejecutarse de manera comprimida y simultánea durante las primeras semanas de noviembre, requiriendo un sobreesfuerzo del desarrollador para asegurar la estabilidad del sistema antes de la fecha de entrega final el 26 de noviembre.

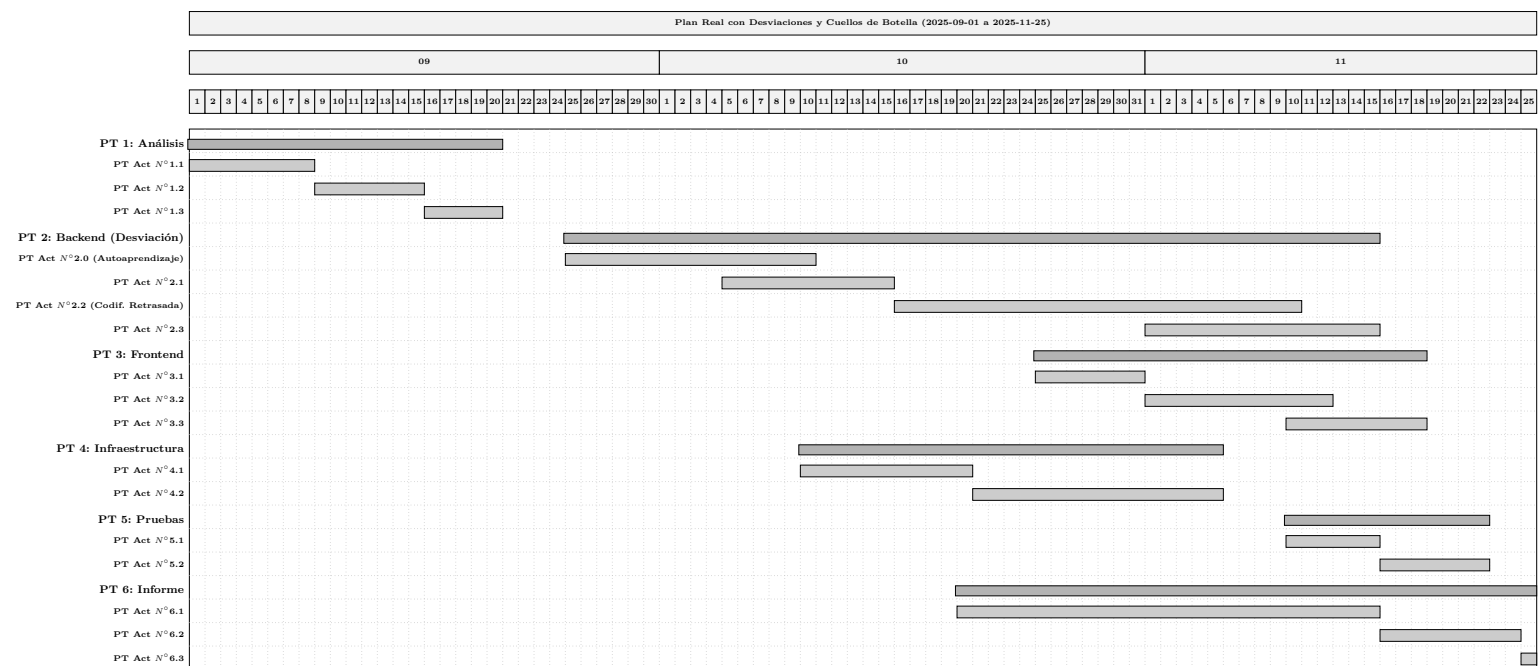


Figura D.1: Carta Gantt con Desviaciones (Ejecución Real)

D.2. Riesgos de Alto nivel (Amenazas), Impacto, estrategia

En el desarrollo del proyecto, se identificaron diversos riesgos que podían comprometer el éxito de la implementación, la calidad del software o el cumplimiento de los plazos. El análisis de riesgos se centró en cuatro dimensiones principales: competencia técnica del equipo, complejidad de la arquitectura distribuida y definición del alcance funcional.

Riesgo (Amenaza)	Impacto / Nivel	Estrategia de Mitigación (Acción)	¿Se materializó?
1. Curva de aprendizaje en nuevas tecnologías (Go/Fiber) El equipo carece de experiencia previa profunda en el lenguaje Go y su arquitectura limpia.	Alto Retrasos críticos en la fase de desarrollo del Backend.	Autoaprendizaje Intensivo y Arquitectura Modular: Se definió una etapa de estudio autodidacta de la documentación oficial y se adoptó <i>Clean Architecture</i> para facilitar la detección de errores y el mantenimiento del código.	SÍ
2. Fallos en la Arquitectura de Despliegue (Docker/Túneles) Posible incompatibilidad entre contenedores o fallos en la exposición segura vía Cloudflare Tunnel.	Medio Inaccesibilidad del sistema desde redes externas.	Validación Temprana en Entorno Controlado: Se implementó el servidor físico y la configuración de Docker en las primeras etapas (semanas 1-2) para validar la conectividad antes de desarrollar la lógica compleja.	NO
3. Sobrecarga de métricas y latencia La recolección de datos en tiempo real podría saturar la base de datos o ralentizar el dashboard.	Medio Mala experiencia de usuario (UX).	Uso de InfluxDB y Optimización: Se seleccionó una base de datos de series temporales específica para alto volumen de escritura y se definieron intervalos de muestreo configurables (10s) para evitar cuellos de botella.	NO
4. Alcance indefinido Intentar abarcar demasiados tipos de dispositivos (Switches, Routers, IoT) simultáneamente.	Medio Software complejo y difícil de estabilizar.	Delimitación Estricta del Alcance: Se estableció formalmente en los requisitos que la versión 1.0 solo soportaría Servidores y Estaciones de Trabajo Linux, dejando otros dispositivos para trabajo futuro.	NO

Tabla D.1: Matriz de Riesgos, Estrategias y Estado Final

Como se observa en la Tabla D.1, los riesgos técnicos relacionados con la infraestructura (Riesgo 3) y el rendimiento (Riesgo 4) fueron mitigados exitosamente gracias a las decisiones arquitectónicas tempranas. Sin embargo, los riesgos asociados al factor humano y temporal (Riesgos 1 y 2) se materializaron, exigiendo un esfuerzo adicional en las etapas finales del proyecto.

D.3. Estimación CU

Tipo de caso de uso	Cantidad de CU	Cálculo ponderado
5 Simple	3 transacciones o menos 15	$5 \times 15 = 75$
10 Medio	4 a 7 transacciones 6	$10 \times 6 = 60$
15 Complejo	Más de 7 transacciones 0	$15 \times 0 = 0$
Total de CU: 21		135

Tabla D.2: Cálculo estimación tipo de caso de uso.

Tipo de actor	Descripción	Cant.	Cálculo pond.
1 Simple	Otro sistema que interactúa con el sistema mediante API (Agente).	1	$1 \times 1 = 1$
2 Medio	Otro sistema vía protocolo o interfaz texto.	0	$2 \times 0 = 0$
3 Complejo	Persona interactuando mediante GUI (Equipo TI, Encargado).	2	$3 \times 2 = 6$
UAW			7

Tabla D.3: Cálculo estimación tipo de actor.

Calcular UUCP (Unadjusted Use Case Point)

$$UUCP = UAW + UUCW = 7 + 135 = 142$$

Calcular TCF (Technical Complexity Factor)

$$TCF = 0,6 + (0,01 \times 59) = 1,19$$

Calcular EF (Environmental Factor)

$$EF = 1,4 + (-0,03 \times 23) = 0,71$$

Cálculo UCP Final

$$UCP = 142 \times 1,19 \times 0,71 = 119,97 \approx 120$$

Estimación Teórica (LOE)

$$LOE = 20 \text{ hrs} \times UCP$$

$$LOE = 20 \times 120 = 2,400 \text{ hrs.}$$

Evaluación de relevancia de factores técnicos y ambientales	
Irrelevante	De 0 a 2
Medio	De 3 a 4
Esencial	5

Tabla D.4: Evaluación de relevancia de factores técnicos.

Calculate TCF (Technical Complexity Factor)

Technical Factor	Mult.	Relevancia	Resultado
Distributed System	2	5	10
Application performance objectives	1	3	3
End-user efficiency (on-line)	1	4	4
Complex internal processing	1	4	4
Reusability	1	4	4
Installation ease	0,5	4	2
Operational ease, usability	0,5	4	2
Portability	2	5	10
Changeability	1	5	5
Concurrency	1	4	4
Special security features	1	5	5
Provide direct access for third parties	1	3	3
Special user training facilities	1	3	3
Total			59

Tabla D.5: Factores de complejidad técnica.

Cálculo TCF:

$$TCF = 0,6 + (0,01 \times 59) = 1,19$$

Environmental Factor	Mult.	Relevancia	Resultado
Familiar with Objectory	1,5	4	6
Application experience	0,5	3	1,5
Object Oriented experience	1	5	5
Analyst capability	0,5	5	2,5
Motivation	1	5	5
Stable requirements	2	4	8
Par time workers	-1	1	-1
Difficult programming language	-1	4	-4
Total			23

Tabla D.6: Factores ambientales.

Cálculo EF:

$$EF = 1,4 + (-0,03 \times 23) = 0,71$$

Level of Effort. Schneider y Winters proponen que: Si la suma entre el número de factores de entorno (F1 a F6) inferiores a 3 y el número de factores de entorno (F7 a F8) superiores a 3:

- Es menor o igual a 2 entonces $LOE = 20$.
- Es 3 o 4 $LOE = 28$.
- Es mayor a 4 reconsiderar el proyecto.

Análisis de Riesgo en Nocturne Scope:

Factores F1-F6 < 3: **0**

Factores F7-F8 > 3: **1** (Solo F8 Lenguaje difícil = 4)

Suma Total = 1.

Conclusión: Como $1 \leq 2$, se utiliza **LOE = 20**.

D.4. Resumen Esfuerzo

El final de este documento indica las horas reales destinadas en realizar cada una de las fases del desarrollo del software, correspondientes a la suma de las horas gastadas por el equipo.

Actividades/fases/casos de Uso	N° Horas
Cuántas horas se dedicaron en aprendizaje y capacitación (Go, Fiber)	100
Cuántas horas se dedicaron en diseño de arquitectura	10
Cuántas horas se dedicaron en programación Backend	150
Cuántas horas se dedicaron en programación Frontend y Agentes	200
Cuántas horas se dedicaron en configuración de Docker y Despliegue	20
Cuántas horas se dedicaron en informe completo (preparar y corregir)	30
TOTAL	510

Tabla D.7: Anexo resumen esfuerzo.

Resumen de Level of Effort (Real vs Estimado)

$UCP = 120$

$EsfuerzoReal = 510 \text{ hr}$

Productividad Real:

$510/120 = 4,25 \text{ hrs} \times \text{CU}$

D.5. Retrospectiva Proyecto

Síntesis del porcentaje de cumplimiento de los requerimientos por cada módulo

Considerando los límites definidos en la especificación de requerimientos, donde se estableció que la versión inicial se enfocaría en servidores y estaciones de trabajo (excluyendo dispositivos de red como routers y switches), el cumplimiento de los objetivos ha sido satisfactorio. A continuación, se detalla el cumplimiento por módulo funcional:

Módulo	% Cumplimiento	Justificación
Autenticación y Usuarios	100 %	Se implementó el registro, inicio de sesión y gestión de roles (RF 01).
Agente y Métricas	100 %	Recolección exitosa de telemetría (CPU, RAM, Temperatura) y envío seguro vía API.
Gestión de Dispositivos	100 %	Generación de tokens y vinculación de equipos completada (RF 02, RF 03).
Visualización Topología	100 %	Representación gráfica interactiva de nodos operativa en el Canvas (RF 06).
Dashboard y Monitoreo	100 %	Visualización de series temporales en tiempo real implementada (RF 04).
Automatización	100 %	Ejecución de tareas y alertas configurables lograda (RF 07).
Reportabilidad	100 %	Generación de informes históricos y exportación a PDF funcional (RF 05, RF 08).
Routers y Switches	0 %	Funcionalidad excluida explícitamente del alcance de esta versión (Trabajo Futuro).

Tabla D.8: Porcentaje de cumplimiento de requerimientos por módulo.

Análisis éxito/fracaso del proyecto

El proyecto se considera un **éxito** basándose en las tres dimensiones fundamentales analizadas durante el desarrollo:

- **Éxito Técnico:** Se logró construir una solución funcional que simplifica drásticamente la gestión TI, eliminando configuraciones manuales complejas y operando en tiempo real, cumpliendo así con el objetivo general propuesto.
- **Éxito Económico:** El análisis financiero arrojó un Valor Actual Neto (VAN) positivo de \$47.082.593 CLP, demostrando una alta rentabilidad y viabilidad comercial del producto.
- **Éxito Académico/Profesional:** El desarrollo permitió validar competencias críticas de egreso, destacando la capacidad de autoaprendizaje al dominar tecnologías complejas como Go (Golang) y Clean Architecture durante el proceso.

Conclusión respecto a los resultados

En conclusión, aunque la planificación temporal sufrió alteraciones importantes debido a la adopción de nuevas tecnologías, la gestión ágil del proyecto permitió absorber estos impactos sin comprometer el alcance. La estimación por Casos de Uso fue útil para dimensionar la complejidad funcional, pero inexacta para predecir la duración en un proyecto unipersonal. El resultado final es una plataforma robusta, validada y entregada a tiempo gracias a la mitigación temprana de riesgos arquitectónicos.

D.6. Iteraciones en el desarrollo

El desarrollo del sistema se ejecutó bajo una metodología incremental. A continuación, se detallan las iteraciones realizadas, las funcionalidades liberadas en cada una y la retroalimentación técnica o validación obtenida, basándose en la ejecución real del proyecto.

Iteración / Incremento	Funcionalidad Desarrollada	Fecha	Retroalimentación y Validación
1. Análisis y Diseño	Definición de Requisitos (RF/RNF), Diseño de Base de Datos (MER), Arquitectura de Red y Prototipos de UI (Mockups).	Septiembre (Semanas 1-4)	Validación de la viabilidad técnica del diseño y aprobación de la arquitectura basada en microservicios y contenedores.
2. Núcleo Backend (Adaptación)	Implementación de API REST en Go, Arquitectura Limpia, Autenticación (JWT) y conexión a bases de datos (PostgreSQL/InfluxDB).	Octubre (Semanas 5-9)	Detección de curva de aprendizaje en Go (Riesgo materializado). Se ajustó el cronograma para priorizar la estabilidad del núcleo antes de avanzar.
3. Frontend e Integración	Interfaz web en Next.js, Dashboard de métricas, Visualización de Topología y desarrollo del Agente de recolección para Linux.	Noviembre (Semanas 10-12)	Validación visual de la recepción de datos en tiempo real. Corrección de latencia en la carga de componentes gráficos (cumplimiento de RNF 01).
4. Infraestructura y Estabilización	Despliegue containerizado (Docker), configuración de seguridad (Cloudflare Tunnel) y pruebas integrales de sistema.	Noviembre (Semanas 13-14)	Confirmación de estabilidad operativa en entorno de pruebas. Verificación de la seguridad en la transmisión de datos (SSL/TLS) y cierre exitoso del desarrollo.

Tabla D.9: Resumen de iteraciones, funcionalidades y validaciones del proyecto.